

ANQI: An Approximate Reverse k -NN Query Index at Scale

Yujian Fu*

National University of Singapore
Singapore
yujianfu@comp.nus.edu.sg

Weida Liang*

National University of Singapore
Singapore
weida_liang@u.nus.edu

Anqi Dang*

Nanyang Technological University
Singapore
anqi012@e.ntu.edu.sg

Weng-Fai Wong

National University of Singapore
Singapore
dcswwf@nus.edu.sg

Bingsheng He

National University of Singapore
Singapore
dcsheb@nus.edu.sg

ABSTRACT

Reverse k -nearest-neighbor ($RkNN$) search asks which base vectors would rank a query vector among their own k nearest neighbors. Whether a base vector belongs to the answer depends on the requested rank k and that vector’s local neighborhood. Neighborhoods differ across base vectors, and the requested rank may differ across queries; supporting $RkNN$ at scale therefore makes both query-time computation and threshold storage difficult.

We present ANQI to address both bottlenecks by separating candidate routing from threshold reconstruction. It lifts each object’s reverse cutoff into a Euclidean ANN score. This lets a proximity graph generate candidates without comparing the query with every object, which reduces candidate-generation cost. A compact verifier then reconstructs the requested rank cutoff from a shared low-rank model and quantized residuals, avoiding a dense threshold table while retaining object-specific detail. One graph and verifier serve all materialized target ranks without rebuilding the index. Across five million-scale datasets, ANQI delivers multi-fold throughput gains over recall-qualified baselines at high-recall operating points (recall 0.95 for L_2 and 0.99 for IP). Precision is at least 0.99 at every reported operating point. Beyond these million-scale results, to our knowledge, ANQI is the first $RkNN$ index to demonstrate an industrial-scale billion-vector construction-and-query result: on SIFT1B it achieves recall and precision above 0.95 at more than 10,000 QPS. The implementation is available at <https://github.com/Yujian-Fu/anqi-implementation>.

PVLDB Reference Format:

Yujian Fu, Weida Liang, Anqi Dang, Weng-Fai Wong, and Bingsheng He. ANQI: An Approximate Reverse k -NN Query Index at Scale. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

1 INTRODUCTION

Conventional k -nearest-neighbor (kNN) search is query-centered: given a vector q , it returns the k base vectors closest to q . Reverse k -nearest-neighbor ($RkNN$) search asks whose own top- k list would

*The first three authors contributed equally to this work.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

Forward kNN

Reverse kNN

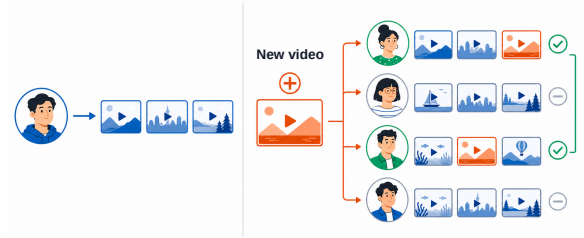


Figure 1: Forward and reverse top- k retrieval in a recommendation workload. Left: forward kNN retrieves the top- k videos for one user. Right: after a new video q arrives, $RkNN$ returns the users whose own top- k lists would include q ; the green users are the returned set.

contain q . This direction supports item-driven retrieval. A seller can find the users most likely to rank a new or discounted item highly [3, 4]; a service can find subscribers for whom a new document is relevant, or customers affected by a new store location [38]. In the recommendation example in fig. 1, forward kNN selects videos for one user, whereas $RkNN$ selects the initial audience for a new video.

Modern embedding collections make the reverse query difficult for two reasons. First, forward proximity does not order reverse candidates. Each base vector judges q against its own neighborhood, so a nearby vector can reject q while a farther one accepts it. A proximity index can therefore miss true reverse neighbors even when it retrieves an accurate forward neighborhood. Second, the acceptance cutoff is specific to both the base vector and the requested k . Computing it during every query is expensive, while storing every cutoff creates a dense object-by-rank table. At million- and billion-vector scale, broader candidate exploration and larger verification state force a direct trade-off between retrieval quality, query efficiency, and memory.

High-dimensional $RkNN$ methods built on proximity graphs use candidate generation to avoid a full scan and then verify the reverse predicate for each candidate. Figure 2(A) shows why both stages are difficult. For a base vector o , let $r_k(o)$ be the distance to its k -th nearest other base vector. In L_2 space,

$$o \in RkNN(q) \iff \delta(q, o)^2 \leq r_k(o)^2. \quad (1)$$

At $k = 6$, the nearby vector A lies in a dense region and rejects q because its acceptance radius is small. The farther vector B lies in a sparser region and accepts q . Base-space distance therefore

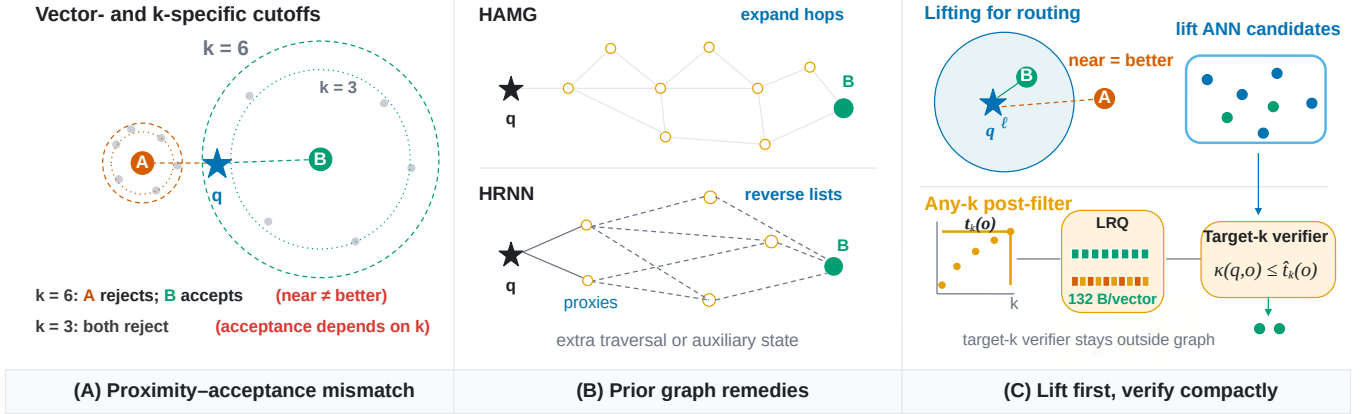


Figure 2: Why forward routing is insufficient and how ANQI changes it. (A) Object- and rank-specific cutoffs can reverse the base-space proximity order. (B) Prior graph systems recover candidates after forward routing. (C) ANQI encodes the cutoff before routing and verifies the target- k predicate.

ranks a rejector ahead of an acceptor. When the request changes to $k = 3$, B also rejects q , illustrating that verification must recover a different cutoff for each rank. Existing systems close these gaps after ordinary routing. HAMG [52] expands multiple graph hops and HRNN [55] follows materialized reverse lists, increasing traversal or auxiliary state. SFT [51] and RDT [13] add range tests or forward- k NN refinement. Inner-product (IP) systems likewise scan bounded user blocks and may invoke item scans or hash searches for surviving users [3, 4, 30].

ANQI is a large-scale approximate Rk NN index that changes candidate routing instead of repairing it afterward. Figure 2(C) shows the design. Proximity lifting embeds each base vector’s acceptance cutoff in its indexed score. A standard norm-equalization step then turns the reverse-aware score into Euclidean proximity, so mature ANN graphs such as HNSW [42], NSG [23], or DiskANN [34] can retrieve reverse candidates. The lifted graph supplies candidate coverage, while an independent target- k verifier makes the final decision. To avoid a dense threshold table, ANQI represents the object-by-rank matrix with a shared low-rank model and packed learned residual quantization (LRQ), as formalized in section 4.2. LRQ preserves the object- and rank-specific detail left after low-rank reconstruction. Only the requested threshold column is decoded, and verifier state never enters graph distance. The resulting graph-search substrate serves both L_2 and IP after the corresponding score transformation, while the separate verifier reduces persistent threshold storage.

Table 1 compares direct high-dimensional reverse-neighbor indexes. “Approx.” denotes methods that trade exact answers for faster query processing. “Any- k ” means that a query selects a materialized target rank from the range supported by one index without rebuilding its candidate graph. Scale is the largest collection evaluated in each cited work; ANQI’s 1B entry refers to the separate SIFT scaling prototype in section 6.4. ANQI uses one graph-and-verifier architecture for L_2 and IP and extends the reported scale of the compared direct indexes from ten million to one billion vectors. Its main contributions are:

- We formulate high-dimensional Rk NN membership under L_2 and IP through a common threshold-MIPS interface. It

Table 1: Capabilities and reported scale of high-dimensional reverse-neighbor indexes.

Method	Index	L_2	IP	Any- k	Approx.	Scale
SFT [51]	Tree	✓	–	✓	✓	10^4
RDT [13]	Tree	✓	–	✓	✓	10^6
HAMG [52]	Graph	✓	–	✓	✓	10^7
HRNN [55]	Graph	✓	–	–	✓	10^6
SIMPFER [3]	Bounds	–	✓	–	–	10^6
SAH [30]	Hashing	–	✓	–	✓	10^6
ANQI	Graph	✓	✓	✓	✓	10^9

encodes each base vector’s reverse decision boundary in the indexed score and enables one Euclidean candidate-search substrate for both predicates.

- We model vector- and rank-specific thresholds with a low-rank representation and LRQ. For L_2 , the Local Radius Law provides a Euclidean rationale for shared rank structure; for IP, the same representation is fitted directly to signed thresholds. Residual codes recover boundary fidelity without a full-precision threshold table.
- We evaluate the integrated system on five million-scale datasets under both predicates. ANQI achieves $2.14\text{--}4.44\times$ the throughput of the fastest recall-qualified L_2 baseline and $10\text{--}2,900\times$ that of the fastest recall-qualified IP baseline at operating points selected for high recall (near 0.95 for L_2 and 0.99 for IP). At these points, ANQI’s precision is at least 0.997 across the 15 reported dataset–target-rank configurations. An industrial-scale SIFT1B prototype demonstrates both construction and query processing, with both recall and precision above 0.95 at more than 10,000 QPS.

2 RELATED WORK

ANQI sits at the intersection of proximity graph search, reverse-neighbor indexing, dimension lifting, and compact neighbor-distance modeling.

Proximity graph indexes. High-dimensional ANN search spans locality-sensitive hashing [5, 18, 31], adaptive tree indexes [46], and quantized billion-scale engines [36]. Proximity graphs instead route

a query through sparse data-dependent neighborhoods. Early graph search used approximate k NN graphs [28]; NN-Descent [19] established scalable graph refinement, and EFANNA [22] combined tree seeding with graph search. FANNG [29] and PANNG/ONNG [32, 33] developed graph pruning, while HNSW [42] and NSG [23] developed navigable forward-ANN graphs. Vista [24] adapts connectivity to imbalanced data, DiskANN/Vamana [34] targets billion-scale search, and ParlayANN [44] provides parallel construction. Graph range retrieval returns a variable-cardinality set but starts from one query radius [43]. ANQI adopts a bounded-degree graph as its forward-search engine after lifting. It does so through a reverse-aware geometry that makes graph traversal useful for RkNN candidates while retaining mature forward-ANN construction and navigation.

Reverse- k NN indexes. Reverse-neighbor search began as an influence query and later gained exact spatial and metric indexes [2, 38, 54]. Methods for high-dimensional data commonly separate candidate generation from vector-specific verification. SFT [51] starts from forward neighbors. Arthur–Oudot address reverse 1-NN through lifted LSH subproblems [6]. RDT [13] combines local tests with forward- k NN refinement. Graph systems broaden candidate coverage in the base space. HAMG [52] uses multi-hop expansion, while HRNN [55] uses materialized reverse lists. HRNN is our closest direct competitor. Reverse-MIPS methods specialize the predicate to inner product. SIMPFER [3, 4] prunes candidates with per-user score bounds. Concurrent RT-RkNN [10] solves an exact two-dimensional spatial variant with GPU ray tracing. Its setting differs from high-dimensional vector search. ANQI retains the same two-stage decomposition. Its distinction is to incorporate reverse thresholds into candidate routing and to use one architecture for both L_2 and inner product.

Dimension lifting. The L_2 reverse predicate is a point-enclosure query over balls $B(o, r_k(o))$. Its signed power distance, $\|q - o\|^2 - r_k(o)^2$, belongs to the classical geometry of power diagrams and sphere-to-halfspace lifting [7]. Cheong et al. exploit related geometry for exact fixed-dimensional reverse-neighbor search, while Arthur–Oudot construct lifted LSH subproblems for high-dimensional reverse 1-NN [6, 17].

ANQI builds on this geometric lineage but targets a different high-dimensional index interface. It derives the scalar correction from a rank-dependent local-neighbor cutoff, builds one approximate graph for candidate generation, and applies compact target- k verification. SAH [30] is the closest transform-based reverse-MIPS design. It combines a shifting-aware transform with hashing and a cone tree.

Graph indexes for MIPS. Maximum-inner-product search first motivated asymmetric hashing reductions [47, 49, 50, 56]. Graph-based methods optimize transformations, construction, or navigation [14, 15, 40, 45, 53, 58, 59]. They can serve as candidate backbones for query-centered top- k MIPS, but they do not encode the rank-dependent acceptance boundary of each base vector for reverse reporting.

Neighbor-distance modeling and compression. Prior work models vector-specific k -distance curves to reduce storage or support a query-time rank. Achtert et al. fit a log-log power model [1]. This

approximation is distinct from their exact metric-tree method [2]. Berrendorf et al. learn compact bounds with safe refinement [11]. Obermeier et al. use nonlinear approximation [48], and HAMG [52] extrapolates local distances during pruning. These results motivate compact modeling of rank-dependent neighbor thresholds. Low-rank approximation and vector quantization provide complementary compression tools [8, 16, 20, 26, 27, 35, 37, 57]. ANQI applies these principles to the vector-by-rank matrix of scalar thresholds. Its verifier combines a low-rank model with learned packed residual quantization. This representation avoids a dense full-precision threshold table. The model coefficients and quantized codes participate only in verification and never in graph routing. The result is a compact verifier integrated with a reverse-aware shared candidate geometry.

3 PROBLEM FORMULATION

Let $\mathcal{X} = \{o_1, \dots, o_n\} \subset \mathbb{R}^D$ be a collection of n base vectors. We use $\langle \cdot, \cdot \rangle$ for inner product, $\|\cdot\|$ for Euclidean norm, and $\delta(x, y) = \|x - y\|$ for Euclidean distance.

Definition 1 (Reverse k -NN). For a query $q \in \mathbb{R}^D$ and requested rank k , let $r_k(o)$ be the distance from $o \in \mathcal{X}$ to its k -th nearest vector in $\mathcal{X} \setminus \{o\}$, and let $\text{ip}_k(o)$ be its k -th largest inner product with a vector in $\mathcal{X} \setminus \{o\}$. Threshold ties are included. The reverse- k NN answer $A_k(q)$ contains every base vector for which q is among the k best matches. Under L_2 distance,

$$A_k(q) = \text{RkNN}(q) = \{o \in \mathcal{X} : \delta(q, o)^2 \leq r_k(o)^2\}. \quad (2)$$

Under inner product,

$$A_k(q) = \text{RkNN}(q) = \{o \in \mathcal{X} : \langle q, o \rangle \geq \text{ip}_k(o)\}. \quad (3)$$

The answer size is data-dependent and need not equal k .

One predicate for both metrics. The two definitions differ only in the direction of their native scores. We orient both as a smaller-is-better threshold test so that both metrics share one predicate and query interface:

$$(\kappa(q, o), t_k(o)) = \begin{cases} (\delta(q, o)^2, r_k(o)^2), & L_2, \\ (-\langle q, o \rangle, -\text{ip}_k(o)), & \text{inner product.} \end{cases} \quad (4)$$

Thus, in either metric, the shared membership test is $o \in A_k(q) \iff \kappa(q, o) \leq t_k(o)$. This notation does not change the query; it only gives the two reverse predicates a common form.

Approximate indexed query. Evaluating this predicate for every base vector requires a full scan. An approximate index avoids this cost with two stages. Given (q, k) , candidate generation retrieves a small set $C(q) \subseteq \mathcal{X}$ that should contain the true reverse neighbors. Verification then tests only these candidates. It uses a represented cutoff $\hat{t}_k(o)$ and returns

$$\hat{R}_k(q) = \{o \in C(q) : \kappa(q, o) \leq \hat{t}_k(o)\}. \quad (5)$$

This decomposition creates two independent error paths. A true reverse neighbor that is missing from $C(q)$ cannot be recovered by verification. This error lowers recall. An inaccurate $\hat{t}_k(o)$ can reject a true answer or accept a false one. This error affects recall or precision. The returned set is exact only when $C(q)$ covers $A_k(q)$

Table 2: Principal notation used throughout the paper. The unified predicate is $\kappa(q, o) \leq t_k(o)$; hats denote quantities reconstructed by the compact verifier.

Symbol	Meaning
$\mathcal{X}, n, D$	Base-vector collection, cardinality, and dimension.
$q, o, k, \mathcal{K}$	Query, base vector, target rank, and represented rank set.
$\kappa(q, o), t_k(o)$	Oriented query score and exact rank-specific cutoff.
$\hat{t}_k(o)$	Reconstructed rank-specific cutoff.
$A_k(q), C(q), \hat{R}_k(q)$	Exact answer, candidate set, and returned answer.
T, Θ, M, b	Threshold matrix, verifier state, basis rank, and residual width.
K_{pivot}, G, L	Construction rank, graph degree, and descent width.
$\mathcal{G}, \mathcal{E}, \mathcal{V}, \mathcal{I}$	Graph, entry structure, verifier state, and index.
$Q, E_{\text{graph}}(q)$	Query batch and graph-distance evaluations per query.

and $\hat{t}_k(o) = t_k(o)$ for every candidate. We therefore measure the two errors separately.

Evaluation protocol. For a query collection Q , we report retrieval quality against online and offline cost. Online cost consists of QPS and graph-distance evaluations; offline cost consists of construction time and persistent storage. Let $E_{\text{graph}}(q)$ count the graph-space distance evaluations made while traversing the index. We report their average per query as

$$\bar{E}_{\text{graph}} = \frac{1}{|Q|} \sum_{q \in Q} E_{\text{graph}}(q). \quad (6)$$

This is the average number of graph-distance evaluations per query. Verifier reads and threshold preprocessing are excluded because they are not graph distance evaluations; QPS is measured separately under section 6. On the canonical million-scale datasets, exhaustive base-to-base computation supplies exact thresholds and exact RkNN answers. We report recall to measure recovered true answers and precision to expose extra returned vectors.

Let $Q_+ = \{q \in Q : |A_k(q)| > 0\}$. For $q \in Q_+$, write $A = A_k(q)$ and $\hat{R} = \hat{R}_k(q)$. The outcome counts are

$$\text{TP}_q = |A \cap \hat{R}|, \quad \text{FP}_q = |\hat{R} \setminus A|, \quad \text{FN}_q = |A \setminus \hat{R}|. \quad (7)$$

The per-query quality measures are

$$R_q = \frac{\text{TP}_q}{\text{TP}_q + \text{FN}_q}, \quad P_q = \frac{\text{TP}_q}{\text{TP}_q + \text{FP}_q}. \quad (8)$$

We define MacroR and MacroP as the means of R_q and P_q , respectively, over $q \in Q_+$. If $\hat{R}_k(q)$ is empty, we set $P_q = 1$ because it contains no false positive; recall and F1 are zero. F1 is computed per query and then averaged. Macro averaging gives each query equal weight, regardless of reverse-answer-set size.

4 ANQI

ANQI is a two-stage reverse-neighbor index. Proximity lifting moves the object-specific reverse cutoff into the indexed representation, so a forward ANN graph can generate high-coverage candidates. A compact verifier then reconstructs the requested target- k cutoff and filters those candidates. Figure 3 summarizes index construction and query processing. Complete proofs and supplementary bounds for the mathematical statements in this section are collected in Appendix A.

4.1 Lifting for Candidate Generation

Forward- k NN graph indexes such as HNSW [42], NSG [23], and DiskANN [34] provide a mature query-centered retrieval substrate. Applying them directly to reverse search fails because native distance or similarity gives one ordering around q , whereas reverse membership depends on a different acceptance cutoff for every base vector. ANQI makes that cutoff part of the indexed vector, so objects likely to accept q can be visited early by a forward traversal. For fixed k , expanding the L_2 predicate gives

$$\begin{aligned} \|q - o\|^2 \leq r_k(o)^2 &\iff \|q\|^2 + \|o\|^2 - 2 \cdot \langle q, o \rangle \leq r_k(o)^2 \\ &\iff \langle q, o \rangle - \frac{1}{2} \cdot (\|o\|^2 - r_k(o)^2) \geq \frac{1}{2} \cdot \|q\|^2. \end{aligned} \quad (9)$$

The left side combines the query-object interaction with the acceptance radius of o , while the right side is identical for every object considered by the query. The varying radii can therefore influence candidate order instead of being consulted only after candidate discovery. Inner product admits the same separation:

$$\langle q, o \rangle \geq \text{ip}_k(o) \iff \langle q, o \rangle - \text{ip}_k(o) \geq 0. \quad (10)$$

In both metrics, the reverse predicate compares an object-side score with a query-only cutoff. The score is the native query-object interaction plus a scalar correction known at construction time. ANQI stores that correction as one additional coordinate and pairs it with a constant query coordinate:

$$o_k^\ell = \begin{cases} [o, -\frac{1}{2} \cdot (\|o\|^2 - r_k(o)^2)], & L_2, \\ [o, -\text{ip}_k(o)], & \text{inner product}, \end{cases} \quad q^\ell = [q, 1]. \quad (11)$$

Let $s_k(q, o) = \langle q^\ell, o_k^\ell \rangle$. Equations (9) and (10) then give

$$\begin{aligned} o \in A_k(q) &\iff s_k(q, o) \geq \tau(q), \\ \tau(q) &= \begin{cases} \frac{1}{2} \cdot \|q\|^2, & L_2, \\ 0, & \text{inner product}. \end{cases} \end{aligned} \quad (12)$$

The lifted inner product reproduces the reverse-aware score without an object-specific query-time operation. The vector-specific threshold is part of the indexed score, and reverse membership becomes a forward maximum-inner-product test.

We use the standard XBOX reduction [9] to expose the reverse-aware score to Euclidean ANN search. The reverse-specific step is the preceding threshold lift, which places each object’s rank-dependent acceptance cutoff in the routing geometry. Choose $M_\star \geq \max_{o \in \mathcal{X}} \|o_k^\ell\|$ and define

$$\bar{o}_k = \left[o_k^\ell, \sqrt{M_\star^2 - \|o_k^\ell\|^2} \right], \quad \bar{q} = [q^\ell, 0]. \quad (13)$$

Every $\bar{o}_k$ has norm $M_\star$, and

$$\|\bar{q} - \bar{o}_k\|^2 = \|q^\ell\|^2 + M_\star^2 - 2 \cdot s_k(q, o). \quad (14)$$

A larger reverse-aware score is therefore exactly equivalent to a shorter Euclidean distance. The membership test becomes

$$o \in A_k(q) \iff \|\bar{q} - \bar{o}_k\|^2 \leq \underbrace{\|q^\ell\|^2 + M_\star^2}_{\rho_k(q)^2} - 2 \cdot \tau(q). \quad (15)$$

THEOREM 1 (EXACT REVERSE-AWARE CANDIDATE GEOMETRY). Assume exact target- k thresholds and real arithmetic. For either L_2

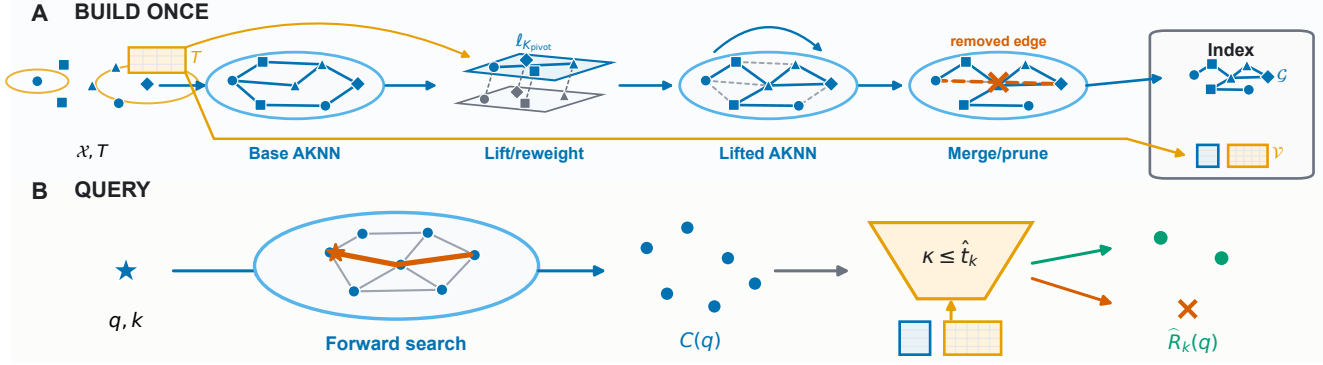


Figure 3: ANQI workflow. Construction turns a base-space AKNN warm start into a lifted graph while independently encoding the threshold verifier. At query time, the graph returns $C(q)$ and the verifier produces $\hat{R}_k(q)$.

or inner product, the construction above has three properties. First, the lifted ball test in (15) is equivalent to exact reverse membership. Second, ordering base vectors by increasing $\|\bar{q} - \delta_k\|$ is equivalent to ordering them by decreasing reverse-aware score $s_k(q, o)$. Third, filtering any candidate set $C(q)$ in the lifted space returns exactly $C(q) \cap A_k(q)$.

The proof substitutes the lifted score into the equal-distance identity and is given in Appendix A.1.

The lift replaces many vector-centered acceptance balls with one query-centered ordering that a forward ANN graph can navigate. It preserves the target- k predicate for candidates reached by the graph, while the verifier makes the final membership decision.

4.2 Compact Threshold Modeling

Verification needs one object-specific threshold at the requested rank, but storing a full-precision scalar for every object and rank creates a dense matrix. ANQI exploits the structure shared across rank columns and reserves a small residual code for the detail that the shared model does not capture.

Euclidean structure. For Euclidean data, the threshold is $t_k(o) = r_k(o)^2$. Classical k NN density estimation and nearest-neighbor limit theory relate the mass enclosed by the k th-neighbor ball to its volume [12, 25, 41]. Intrinsic-dimension estimators use the same local geometry on a smooth data manifold [21, 39]. In a neighborhood where sampling intensity and intrinsic dimension are approximately constant, write $N_o(r) = \rho(o) \cdot V_{d(o)} \cdot r^{d(o)}$. Here $N_o(r)$ is the expected local neighbor mass, $\rho(o)$ is local intensity, and $d(o)$ is local intrinsic dimension. The following rank-radius relation is then algebraic rather than a fitted curve.

PROPOSITION 1 (EUCLIDEAN LOCAL RADIUS LAW). *Under the local ball-mass model,*

$$r_k(o)^2 = \gamma(o) \cdot k^{v(o)}, \quad v(o) = \frac{2}{d(o)}, \quad \gamma(o) = [\rho(o) \cdot V_{d(o)}]^{-v(o)}. \quad (16)$$

For squared thresholds, local density enters through the inverse power $\rho(o)^{-2/d(o)}$, while rank enters through $k^{2/d(o)}$. Equivalently,

$$\log r_k(o) = \frac{1}{d(o)} [\log k - \log(\rho(o) \cdot V_{d(o)})].$$

The log-radius slope is the inverse local dimension over a scale where the model holds. Different objects may have different slopes and intercepts, which explains why one global radius schedule is insufficient. Writing $v(o) = v_0 + \beta(o)$ yields the separable expansion

$$r_k(o)^2 = \sum_{m=0}^{\infty} \underbrace{\frac{\gamma(o) \cdot \beta(o)^m}{m!}}_{f_m(o)} \underbrace{k^{v_0} \cdot (\ln k)^m}_{\phi_m(k)}. \quad (17)$$

The shared functions $\phi_m(k)$ and object-specific coefficients $f_m(o)$ show why nearby rank columns can be represented by a small common basis. For a degree- p truncation, the omitted term is bounded by

$$|R_p(o, k)| \leq \gamma(o) \cdot k^{v_0} \cdot e^{|\beta(o) \cdot \ln k|} \cdot \frac{|\beta(o) \cdot \ln k|^{p+1}}{(p+1)!}. \quad (18)$$

Thus a rank interval with moderate $|\beta(o) \cdot \ln k|$ admits a small shared basis even when the ambient dimension D is large. The assumption is local in scale: curvature, changing density, or a changing intrinsic dimension may alter the power law over a wider interval. The complete derivation appears in Appendix A.2.

This analysis maps naturally to the three terms of the verifier. The mean μ_k captures the collection-wide rank trend, the coefficients $c_m(o)$ modulate a shared basis according to object-specific neighborhood structure, and a small residual records departures that a finite basis cannot reproduce. The residual is therefore part of the design rather than an assumption that the Local Radius Law fits every object exactly.

This structure motivates ANQI’s compact representation. Prior RkNN indexes compress k -distance curves with functional or learned approximations [1, 11, 48], while HAMG [52] estimates local neighbor distances during query processing. ANQI instead encodes the exact vector-by-rank threshold matrix $T[o, k] = t_k(o)$ for independent verification:

$$t_k(o) = \mu_k + \sum_{m=1}^M c_m(o) \cdot g_{m,k} + e_{o,k}. \quad (19)$$

The shared basis $g_{m,k}$ captures the regular rank variation suggested above. Each object stores M coefficients $c_m(o)$ and a packed code for the remaining scalar $e_{o,k}$, which we call learned residual quantization (LRQ). Before a query batch, ANQI decodes only the requested threshold column. For each rank, the ordered codebook $\mathcal{B}_k$ includes

a lower endpoint no larger than every encoded residual, and the raw floor decoder is

$$\begin{aligned} \tilde{e}_{o,k} &= \max\{b \in \mathcal{B}_k : b \leq e_{o,k}\}, \\ \hat{t}_k(o) &= \mu_k + \sum_{m=1}^M c_m(o) \cdot g_{m,k} + \tilde{e}_{o,k}. \end{aligned} \quad (20)$$

Verification then needs one scalar read per candidate. The deployed configuration uses $M = 8$ and learned u8 residual quantization. A standalone control sets $e_{o,k} = 0$ and fits an empirical M-only basis to isolate the shared term in section 6.5.

Inner-product thresholds. Inner product has no radius-volume relation. Its thresholds instead form directional score quantiles, and ANQI applies the same low-rank-plus-LRQ layout directly to the exact signed threshold matrix.

PROPOSITION 2 (PROJECTION QUANTILES FOR INNER PRODUCT). *For a nonzero object o , let $u_o = o/\|o\|$, and let $\pi_k(o)$ be the k th largest value in $\{\langle x, u_o \rangle : x \in \mathcal{X} \setminus \{o\}\}$. Then*

$$\text{ip}_k(o) = \|o\| \cdot \pi_k(o), \quad o \in A_k(q) \iff \langle q, u_o \rangle \geq \pi_k(o). \quad (21)$$

Equivalently, the signed verifier row is $T[o, k] = -\|o\| \cdot \pi_k(o)$. If $o = 0$, the row is identically zero.

The proof factors the positive scalar $\|o\|$ from each inner product; see Appendix A.3.

Across ranks, each signed IP row is the negative upper-quantile curve of projections onto u_o . ANQI learns a shared basis directly from the exact signed threshold matrix, and LRQ stores the remaining object- and rank-specific deviation. The identity does not impose a Euclidean radius law on IP; it identifies the exact directional quantity that the empirical model compresses. Object norm determines the row scale, while projection order determines how the cutoff changes with rank. Thus L_2 and inner product share the same stored layout and decoder for different structural reasons: local ball growth motivates the Euclidean basis, whereas correlated directional quantiles are learned directly for inner product. In both cases, LRQ compresses exact construction-time thresholds rather than substituting a different membership predicate.

4.3 Verification Guarantees and Controls

The reconstructed cutoff determines which type of verification error is possible. Let $\hat{A}_k(q) = \{o : \kappa(q, o) \leq \hat{t}_k(o)\}$ denote the answer obtained by applying the represented threshold to the full base collection.

PROPOSITION 3 (ONE-SIDED THRESHOLD GUARANTEES). *Assume exact target thresholds. If $\hat{t}_k(o) \leq t_k(o)$ for every o , then $\hat{A}_k(q) \subseteq A_k(q)$ and verification introduces no false positives. Its false negatives are confined to*

$$\{o : \hat{t}_k(o) < \kappa(q, o) \leq t_k(o)\}. \quad (22)$$

If $\hat{t}_k(o) \geq t_k(o)$ for every o , then $A_k(q) \subseteq \hat{A}_k(q)$ and verification introduces no false negatives.

The proof chains the represented and exact inequalities with $\kappa(q, o)$; see Appendix A.3.

By (20), the raw floor-decoded threshold satisfies $\hat{t}_k(o) \leq t_k(o)$ in real arithmetic. Before any optional query-time threshold adjustment, LRQ is therefore precision-oriented: its compression error

cannot introduce false positives, while its boundary band can lower recall. The codebook width controls the size of that band. With exact decoded thresholds, verifier retention is one and end-to-end recall depends only on whether graph traversal reaches the exact members.

For L_2 , the local ball-mass model also turns a restrictive relative threshold error ϵ_k into the pooled-recall bound $\text{Recall}_{\text{pool}} \geq (1 - \epsilon_k)^{d_{\text{max}}/2}$. This bound concerns verifier retention; graph coverage is measured separately. Its derivation and the effect of LRQ codebook spacing appear in Appendices A.4 and A.5.

With residuals disabled, the M-only control reconstructs thresholds from the shared basis alone. Section 6.5 measures how much Euclidean rank structure this term captures before the one-sided residual correction.

4.4 Graph Reuse Across Target Ranks

After defining the verifier, ANQI chooses a construction rank K_{pivot} for the proximity graph and uses the represented threshold $t_{K_{\text{pivot}}}(o)$ in each lifted object. This construction rank is distinct from the target rank k requested by a query. It affects how one graph is reused, not how the target-rank cutoff is modeled; the reuse follows from the monotonicity of reverse answer sets.

PROPOSITION 4 (SHARED-RANK CANDIDATE ENVELOPE). *For exact thresholds in either base space and any $K \geq k$,*

$$A_k(q) \subseteq A_K(q). \quad (23)$$

Consequently, the exact lifted acceptance region at K is a candidate superset for every target rank no larger than K .

Neighbor radii are nondecreasing in k , whereas the k th inner-product order statistic is nonincreasing. The proof is in Appendix A.1. For each represented $k \leq K_{\text{pivot}}$, the construction-rank lift therefore supplies a shared candidate envelope, and the target- k verifier removes its extra members.

For the evaluated L_2 cross-pivot setting, query-time collection substitutes the decoded target- k cutoff while retaining the graph edges.

LEMMA 1 (TARGET-RANK SCORE CORRECTION). *Let $s_{\tilde{t}_k}(q, v) = \langle q, v \rangle - \frac{1}{2} \cdot (\|v\|^2 - \tilde{t}_k(v))$. For every target rank k ,*

$$s_{K_{\text{pivot}}}(q, v) + \frac{1}{2} \cdot (\tilde{t}_k(v) - t_{K_{\text{pivot}}}(v)) = s_{\tilde{t}_k}(q, v). \quad (24)$$

Setting $\tilde{t}_k = t_k$ recovers the exact target- k predicate, including when $k > K_{\text{pivot}}$.

Substitution cancels the construction-rank threshold; the proof is in Appendix A.1. A finite graph traversal must still reach the candidates admitted by the corrected envelope. For candidate set $C_k(q)$ and returned set $\hat{R}_k(q) \subseteq C_k(q)$, end-to-end recall separates as

$$\text{Recall}_q = \underbrace{\frac{|A_k(q) \cap C_k(q)|}{|A_k(q)|}}_{\text{candidate coverage}} \cdot \underbrace{\frac{|A_k(q) \cap \hat{R}_k(q)|}{|A_k(q) \cap C_k(q)|}}_{\text{verifier retention}}. \quad (25)$$

The zero-denominator case and full derivation appear in Appendix A.6; the conditional completeness result for exhaustive target-envelope traversal appears in Appendix A.7.

For Euclidean thresholds, the Local Radius Law also describes the added shell beyond the construction rank.

COROLLARY 1 (LOCAL SHELL GROWTH). *Under the local ball-mass model, for $k \geq K$,*

$$\frac{r_k(o)^2}{r_K(o)^2} = \left(\frac{k}{K}\right)^{v(o)}, \quad N_o(r_k(o)) - N_o(r_K(o)) = k - K. \quad (26)$$

Thus the additional local mass is $k - K$, while squared-radius inflation is governed by $v(o) = 2/d(o)$.

The derivation is in Appendix A.2. This result connects cross-pivot candidate growth to local dimension; the measured robustness of the fixed graph is reported in fig. 9.

The constructions above produce the two artifacts used by ANQI. Equations (11) and (13) define graph coordinates $\bar{o}$ and the corresponding query map Λ . Equations (19) and (20) define the compact verifier state. Section 5 shows how construction builds the graph from the first artifact, encodes the second, and combines them only when verifying retrieved candidates.

5 INDEX REALIZATION

The preceding analysis yields two concrete artifacts: reverse-aware coordinates for routing and a compact representation of target-rank thresholds for verification. ANQI turns them into one index without rebuilding a graph from scratch in the lifted space. Construction begins with a base-space AKNN warm start, reweights and refines its neighbor IDs after lifting, and stores the verifier alongside the resulting graph. At query time, algorithm 2 traverses this graph, collects a variable-size candidate set, and applies the requested target- k predicate. Search width controls traversal effort rather than the number of returned vectors.

5.1 From base-space AKNN to the lifted graph

Construction is a warm-started two-space pipeline: base-space AKNN, lifting, lifted-distance reweighting, lifted-space AKNN refinement, and bounded-degree pruning. The second AKNN stage retains the base-space neighbor IDs, recomputes their edge weights in the lifted geometry, and refines them with NN-Descent [19]. The final adjacency is therefore selected in the lifted metric rather than copied from the base-space graph.

Each $\mathcal{W}$ or $\mathcal{H}$ is an n -row neighbor table. The builder seeds $\mathcal{W}_{\text{base}}$ from random-projection-tree leaves, adds reverse edges, and uses beam gathering to retain nearby base-space IDs. After reweighting those IDs in the lifted geometry, each local-join pass pairs sampled neighbors of the same vertex and inserts a pair when it improves either endpoint table. Refinement stops after six passes or when a pass changes fewer than $n/200$ entries. Reverse gathering then forms a candidate pool per vertex, and RNG pruning retains at most G outgoing edges. A separate RP tree maps each query to a leaf medoid used only as an entry point.

The graph uses only the construction-rank coordinate. The coefficient and residual arrays are stored in $\mathcal{V}$ and cannot alter graph distances or edges. One graph can therefore generate candidates for several target ranks while the verifier changes the final decision.

The exact threshold table and trained verifier are construction inputs. On a fixed sample of threshold rows, training starts from a

Algorithm 1 ANQI index construction.

Require: $\mathcal{X}$ (base vectors); $\mathcal{K}$ (represented ranks); $T[o, k]$ (exact thresholds); $K_{\text{pivot}} \in \mathcal{K}$; G (maximum degree); metric m ; trained verifier $\Theta = (\mu, \{g_m\}_{m=1}^M, \{\mathcal{B}_k\}_{k \in \mathcal{K}})$

- 1: **procedure** BUILDINDEX($\mathcal{X}, T, \Theta, \mathcal{K}, K_{\text{pivot}}, G, m$)
- 2: $\mathcal{W}_{\text{base}} \leftarrow \text{APPROXKNN}(\mathcal{X}, m)$ ▷ base-space AKNN warm start
- 3: **for all** $o \in \mathcal{X}$ **in parallel do**
- 4: $z_o \leftarrow \text{REVERSELIFT}(o, T[o, K_{\text{pivot}}], m)$ ▷ (11)
- 5: **end for**
- 6: $(\{\bar{o}\}, \Lambda) \leftarrow \text{EQUALIZENORMS}(\{z_o\})$ ▷ (13)
- 7: $\mathcal{W}_L \leftarrow \text{REWEIGHTWARMSTART}(\mathcal{W}_{\text{base}}, \{\bar{o}\})$ ▷ same IDs; lifted distances
- 8: $\mathcal{H} \leftarrow \mathcal{W}_L$
- 9: **for** $j = 1, \dots, 6$ **do**
- 10: $(\mathcal{H}, \Delta_j) \leftarrow \text{LOCALJOIN}(\mathcal{H}, \{\bar{o}\})$ ▷ NN-Descent updates
- 11: **if** $\Delta_j < n/200$ **then**
- 12: **break**
- 13: **end if**
- 14: **end for**
- 15: $\mathcal{P} \leftarrow \text{ADDREVERSEANDBEAMGATHER}(\mathcal{H}, \{\bar{o}\})$
- 16: $\mathcal{G} \leftarrow \text{REVERSESCATTERANDRNGPRUNE}(\mathcal{P}, G)$
- 17: $\mathcal{E} \leftarrow \text{BUILDRPTREE}(\{\bar{o}\})$
- 18: **for all** $o \in \mathcal{X}$ **in parallel do**
- 19: $c(o) \leftarrow \text{PROJECTROW}(T[o, :] - \mu, \{g_m\})$
- 20: **for all** $k \in \mathcal{K}$ **do**
- 21: $a_{o,k} \leftarrow \mu_k + \sum_{m=1}^M c_m(o) \cdot g_{m,k}$
- 22: $u_{o,k} \leftarrow \text{FLOORENCODE}(T[o, k] - a_{o,k}, \mathcal{B}_k)$
- 23: **end for**
- 24: **end for**
- 25: $\mathcal{V} \leftarrow (\Theta, \{c(o)\}, \{u_{o,k}\})$
- 26: **return** $\bar{I} = (\{\bar{o}\}, \mathcal{G}, \mathcal{E}, \Lambda, \mathcal{V}, K_{\text{pivot}}, m)$
- 27: **end procedure**

rank- M PCA basis and runs three rounds that alternate per-rank one-dimensional Lloyd codebooks with basis refitting. Construction projects every full threshold row onto the learned basis, extends each ordered codebook to the observed residual range, and floor-encodes the residual using (20). The persistent index keeps the coefficients and codes, not the full table T .

5.2 Query processing

Algorithm 2 exposes the two query stages. The decoder first materializes one target-rank threshold column for the whole batch. Each query then uses bounded graph descent to reach the relevant lifted neighborhood. A second frontier follows the connected part of the candidate envelope, and the verifier tests the resulting candidates in the base space.

The two queues have different roles. The bounded queue supplies robust descent even when the entry point lies outside the acceptance region. Once a visited vertex passes the candidate-envelope test, the FIFO follows its connected component without a top- L output cap. For $k \leq K_{\text{pivot}}$, INCANDIDATEENVELOPE applies the shared construction-rank envelope from proposition 4; the target- k verifier removes its extra members. For the evaluated L_2 cross-pivot path, the envelope instead uses

$$s_{K_{\text{pivot}}}(q, v) + \frac{1}{2} \cdot (\hat{t}_k(v) - t_{K_{\text{pivot}}}(v)) \geq \frac{1}{2} \cdot \|q\|^2, \quad (27)$$

This replaces the construction cutoff with $\hat{t}_k(v)$ without changing graph edges, as established by lemma 1. The evaluated IP ranks satisfy $k \leq K_{\text{pivot}}$. Final verification uses the decoded column rather than T ; version stamps implement seen without a per-query clear.

5.3 Computation and storage complexity

Let T_{AKNN} and $T_{\mathcal{E}}$ denote base-space AKNN and entry-tree construction. Lifted refinement evaluates E_{build} vector pairs and performs

Algorithm 2 ANQI batched reverse- k NN search.

Require: I (index); X (original vectors); Q (query batch); $k \in \mathcal{K}$ (requested rank); descent width L

```

1: procedure SEARCHBATCH( $I, X, Q, k, L$ )
2:    $(\hat{t}_k(o))_{o \in X} \leftarrow \text{DECODECOLUMN}(X, k)$  ▷ once per batch
3:   for all  $q \in Q$  in parallel do
4:      $\bar{q} \leftarrow \Lambda(q); e \leftarrow \text{ENTRYPOINT}(E, \bar{q})$ 
5:     initialize bounded queue  $Q_b$ , FIFO  $Q_f$ ,  $C(q)$ , and seen
6:     EXPLORE( $e$ , true)
7:     while  $Q_b$  has an unexpanded item do
8:        $u \leftarrow \text{CLOSESTUNEXPANDED}(Q_b)$ 
9:       for all  $v \in N_G(u)$  do
10:        EXPLORE( $v$ , true)
11:       end for
12:     end while
13:     while  $Q_f$  is not empty do
14:        $u \leftarrow \text{POP}(Q_f)$ 
15:       for all  $v \in N_G(u)$  do
16:        EXPLORE( $v$ , false)
17:       end for
18:     end while
19:      $\hat{R}_k(q) \leftarrow \{o \in C(q) : \kappa(q, o) \leq \hat{t}_k(o)\}$ 
20:   end for
21:   return  $\{\hat{R}_k(q) : q \in Q\}$ 
22: end procedure
23: function EXPLORE( $v$ , descend) ▷ query-local state
24:   if  $v$  is unseen then
25:     mark  $v$ ;  $d_v \leftarrow \|\bar{q} - v\|^2$ 
26:     if descend then
27:       INSERTBOUNDED( $Q_b, (d_v, v), L$ )
28:     end if
29:     if INCANDIDATEENVELOPE( $v, d_v, q, k, I, \hat{t}_k(v)$ ) then
30:       PUSH( $Q_f, v$ );  $C(q) \leftarrow C(q) \cup \{v\}$ 
31:     end if
32:   end if
33: end function

```

P_{build} queue updates of width B_{build} . Given T and a trained Θ , construction costs

$$O\left(T_{\text{AKNN}} + T_E + E_{\text{build}} \cdot D + P_{\text{build}} \cdot \log B_{\text{build}} + n \cdot [D + |\mathcal{K}| \cdot (M + b)]\right). \quad (28)$$

The codebook term assumes binary search over 2^b entries. This bound excludes threshold computation and model fitting. It includes lifted reweighting, refinement, and pruning in E_{build} and P_{build} .

Excluding the source base vectors shared by all methods, persistent index storage is

$$S_{\text{index}} = O\left(n \cdot (D + G + M) + |\mathcal{K}| \cdot M + |\mathcal{K}| \cdot 2^b\right) + n \cdot |\mathcal{K}| \cdot b \text{ bits}. \quad (29)$$

The first term counts machine-word storage for lifted coordinates, adjacency, coefficients, shared model state, and 2^b scalar codewords per codebook. The second is the packed residual stream. The linear entry structure is absorbed, and the dense threshold table is discarded. Decoding one rank costs $O(n \cdot M)$ time and one transient n -scalar column shared by the batch.

For one query, let $S_{\text{adj}}(q)$ be the number of adjacency entries scanned. The online cost is

$$O(E_{\text{graph}}(q) \cdot D + S_{\text{adj}}(q) + E_{\text{graph}}(q) \cdot \log L + |C(q)| \cdot D), \quad (30)$$

where the last term is base-space verification. The threshold-column cost amortizes to $O(n \cdot M/|Q|)$ per query in a batch. A worker uses $O(n + L + |C(q)|)$ reusable query state for the version stamps, bounded queue, and candidate output. In the worst case $E_{\text{graph}}(q)$

and $|C(q)|$ can reach n ; the index is effective when lifted routing keeps both far below a full scan.

6 EXPERIMENTAL EVALUATION

We measure ANQI’s quality-throughput frontier, offline cost, and scaling behavior. Controlled experiments then separate the effects of threshold modeling and graph geometry.

6.1 Experimental setup

The benchmark covers both metrics, several embedding types, and three target ranks. The L_2 datasets span 96-dimensional learned embeddings, 128-dimensional local descriptors, and 960-dimensional GIST vectors. The inner-product datasets contain recommendation and text embeddings. Each main-benchmark dataset has approximately one million base vectors, and every method is tested with held-out queries at $k \in \{10, 50, 100\}$. SIFT1B is evaluated separately at $k = 10$ in the scalability study.

Baselines. For L_2 , SFT [51] filters a forward neighborhood, RDT [13] applies dimensional tests, HAMG [52] explores multiple hops, and HRNN [55] augments graph navigation with ranked and reverse lists. We use the public HRNN codebase and its HNSW-based reference implementations. For IP, SIMPFER [3] uses score bounds; H2-SIMPFER [3] and SA-SIMPFER [3] add transformations; and SAH [30]/SA-Cone [30] combines shifting-aware hashing with a cone tree. We run these methods from the authors’ repository.

Protocol. We retain released defaults and tune only documented quality-cost controls. The recall targets, 0.95 for L_2 and 0.99 for IP, match the high-recall operating ranges of the corresponding baselines; complete frontiers appear in fig. 4. All runs use the same query set, ground truth, and 64 physical cores on two Intel Xeon Platinum 8457C sockets with SMT disabled. We measure QPS over the complete query batch using each released implementation’s native parallel decomposition.

For the five main datasets, exhaustive distance evaluation produces exact base-to-base top-100 thresholds and exact memberships for $k = 10, 50, 100$. ANQI uses one degree-48 graph constructed at $K_{\text{pivot}} = 100$ and one rank-8 packed-u8 verifier; the same index serves all three target ranks. We report macro recall and precision from eq. (8). For L_2 , D counts every online full-vector distance evaluation in candidate generation and verification, which is broader than the graph-distance count in eq. (6).

Table 3: Datasets used in the evaluation. The five million-scale datasets form the main benchmark; SIFT1B is used only for scalability. Distance is the reverse-query measure; n is the number of base vectors, D is the base-space dimension, and $|Q|$ is the number of query vectors.

Dataset	Source	Distance	n	D	$ Q $
SIFT1M	Image	L_2	1,000,000	128	10,000
DEEP1M	Image	L_2	1,000,000	96	10,000
GIST1M	Image	L_2	1,000,000	960	1,000
Music100	Music	IP	999,900	100	100
MSMarco1M	Text	IP	1,000,000	768	9,376
SIFT1B	Image	L_2	10^9	128	100,000

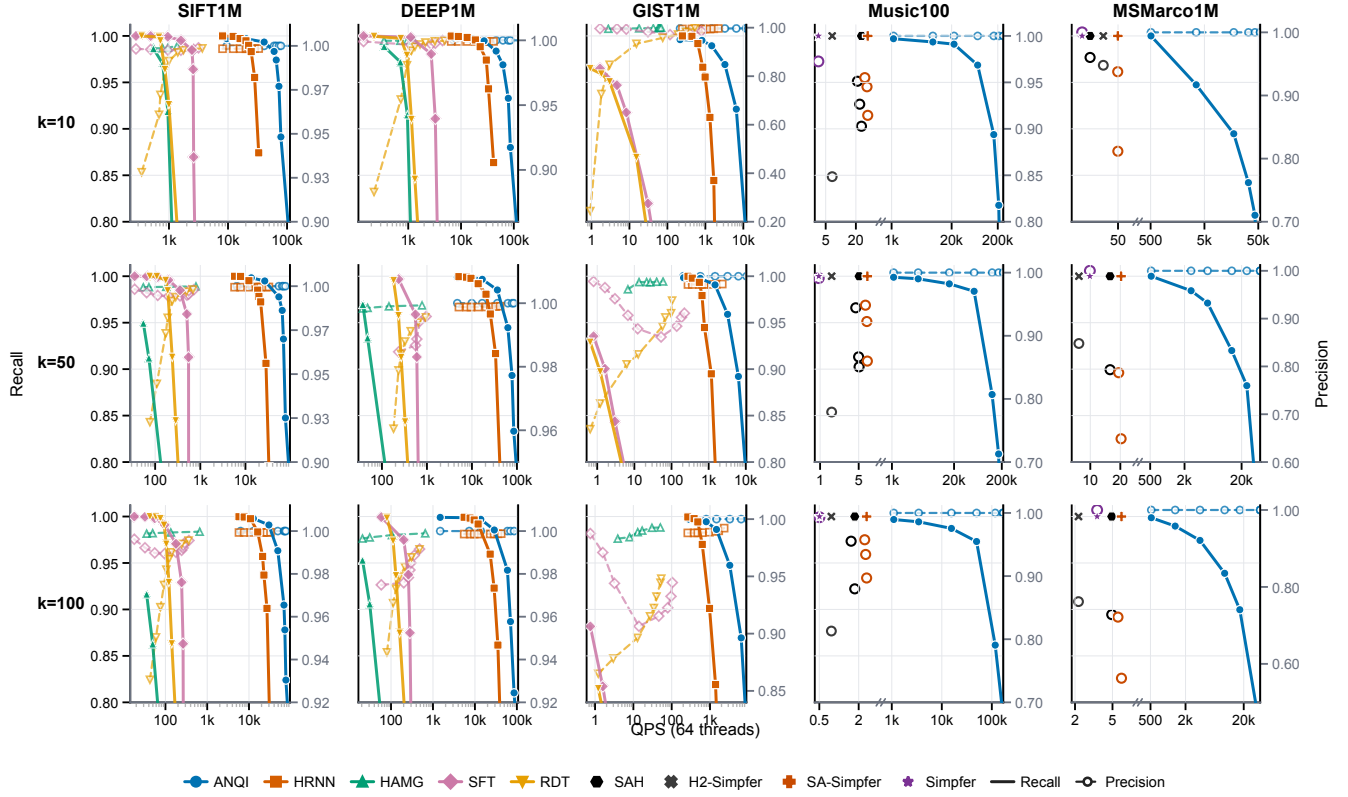


Figure 4: End-to-end recall, precision, and 64-thread QPS on five datasets. Rows are $k = 10, 50, 100$; columns are three L_2 and two IP datasets. Filled traces use the shared leftmost Recall axis, and hollow dashed traces use each panel’s right Precision axis. The IP panels use a marked axis break to omit the empty QPS interval between the low-throughput baselines and ANQI; both displayed segments remain logarithmic. Quality is measured against exact RkNN ground truth.

Table 4: Selected operating points from the complete frontiers in fig. 4, nearest recall 0.95 (L_2) and 0.99 (IP). Cells show recall/precision/QPS/distance evaluations (R/P/Q/D) for L_2 and R/P/Q for IP. Boldface marks the best displayed value in each row and metric (higher is better, except for distance evaluations); red brackets give ANQI’s QPS speedup over the underlined fastest recall-qualified baseline; $\ddagger$ marks a missed target.

L_2 Dataset	k	ANQI	HRNN [55]	HAMG [52]	SFT [51]	RDT [13]
SIFT1M	10	0.954/1.000/71.8k/2.6k [2.57 $\times$]	0.951/0.998/27.9k/5.74k	0.970 /0.999/781/204k	0.964/0.998/2.57k/161k	0.964/0.987/855/349k
	50	0.963 /1.000/65.2k/2.88k [2.84 $\times$]	0.958/0.999/23k/6.75k	0.949 $\ddagger$ /0.999/54.8/4.48M	0.959/0.995/504/1.29M	0.959/0.988/213/1.86M
	100	0.963/1.000/49k/3.3k [2.84 $\times$]	0.957/0.999/17.2k/7.36k	0.917 $\ddagger$ /0.999/35.4/6.96M	0.971 /0.990/179/4.11M	0.970/0.982/105/4.81M
DEEP1M	10	0.955/1.000/69k/2.84k [2.14 $\times$]	0.958/0.999/32.3k/6.85k	0.972/1.000/709/212k	0.981 /0.997/2.73k/253k	0.979/0.981/946/470k
	50	0.958/1.000/55.2k/3.4k [2.15 $\times$]	0.959/0.999/25.6k/8.15k	0.970 /0.998/37.5/6.18M	0.959/0.987/557/1.5M	0.958/0.979/233/2.07M
	100	0.954/1.000/54k/3.74k [2.34 $\times$]	0.960/0.999/23.1k/8.97k	0.953/0.997/20.9/11.2M	0.975 /0.976/201/4.99M	0.974/0.966/112/5.49M
GIST1M	10	0.956/0.997/4.39k/9.07k [4.44 $\times$]	0.956/0.995/987/22.5k	0.724 $\ddagger$ /0.996/2.79/16.6M	0.959 /0.997/2.37/17.8M	0.951/0.847/3.01/31M
	50	0.959/1.000/3.23k/12.4k [4.22 $\times$]	0.967 /0.991/766/31.7k	0.416 $\ddagger$ /0.986/6.70/5.69M	0.935 $\ddagger$ /0.994/0.80/54.1M	0.929 $\ddagger$ /0.835/0.65/107M
	100	0.963 /1.000/2.66k/15.5k [3.86 $\times$]	0.953/0.988/689/31.7k	0.361 $\ddagger$ /0.983/3.92/10.2M	0.882 $\ddagger$ /0.987/0.75/54.4M	0.815 $\ddagger$ /0.865/1.21/47.2M
IP Dataset	k	ANQI	SAH [30]	H2-S [3]	SA-S [3]	SIMP. [3]
Music100	10	0.991/1.000/22.5k [654.2 $\times$]	1.000 /0.903/25.8	1.000 /0.849/6.74	1.000 /0.915/34.5	1.000 /0.973/3.58
	50	0.992/1.000/15.9k [2.215 $\times$]	1.000 /0.851/5.08	1.000 /0.779/1.62	1.000 /0.860/7.18	1.000 /0.991/0.95
	100	0.992/1.000/7.66k [2.895 $\times$]	1.000 /0.880/1.74	1.000 /0.813/0.76	1.000 /0.897/2.65	1.000 /0.993/0.50
MSMarco	10	1.000 /1.000/508 [10.19 $\times$]	1.000 /0.960/32.1	1.000 /0.948/39.6	1.000 /0.812/49.9	1.000 /1.000/28.2
	50	0.998/1.000/973 [47.81 $\times$]	1.000 /0.793/15.8	1.000 /0.848/7.68	1.000 /0.649/20.4	1.000 /1.000/9.90
	100	0.995/1.000/973 [156.2 $\times$]	1.000 /0.728/4.91	1.000 /0.762/2.18	1.000 /0.563/6.23	1.000 /1.000/3.43

6.2 End-to-end effectiveness

Figure 4 shows the complete frontiers; table 4 lists the measured point nearest each recall target from above.

L_2 . Near recall 0.95, ANQI is 2.57–2.84 $\times$ faster than HRNN on SIFT1M, 2.14–2.34 $\times$ faster on DEEP1M, and 3.9–4.4 $\times$ faster on GIST1M. The separation persists as k grows even though the reverse answer sets are larger. Across these nine points, ANQI performs 2.6k–15.5k complete online distance evaluations per query, versus

Table 5: Construction time and index footprint. ANQI includes model fitting; IP baselines use native indexing times and in-memory footprints. Other footprints are persisted. Boldface marks column minima per dataset.

L_2				IP			
Dataset	Method	Build Index (s)	(GB)	Dataset	Method	Build Index (s)	(GB)
SIFT1M	ANQI	102.23	0.85	Music100	ANQI	882.89	0.71
	HRNN [55]	150.00	6.67		SAH [30]	17.73	0.50
	SFT/RDT [13, 51] HNSW	14.00	0.66		H2-Simpfer [3]	1.57	0.84
	HAMG [52]	69.00	20.53		SA-Simpfer [3]	1.45	0.47
DEEP1M	ANQI	88.78	0.73	MSMarco1M	SIMPFER [3]	8.31	0.64
	HRNN [55]	152.00	6.54		ANQI	545.87	3.47
	SFT/RDT [13, 51] HNSW	12.00	0.53		SAH [30]	125.97	0.62
	HAMG [52]	51.17	20.40		H2-Simpfer [3]	5.61	0.86
GIST1M	ANQI	308.17	4.25		SA-Simpfer [3]	5.94	0.47
	HRNN [55]	480.00	10.00		SIMPFER [3]	6.88	0.64
	SFT/RDT [13, 51] HNSW	77.00	3.99				
	HAMG [52]	308.00	23.86				

Table 6: DEEP1M construction and matched-quality query-time breakdown.

Construction: 52.8 s			Query at matched quality					
Component	Time (s)	Share	k	Target	Recall	QPS	Graph μ s	Verify μ s
Base-space AKNN	39.0	74%	10	.95	.96	78k	790	30
Graph build/save	12.1	23%	10	.99	.99	33k	1,900	33
Lift preparation	0.74	1.4%	50	.95	.96	62k	1,000	43
LRQ encode/save	0.25	0.50%	50	.99	.99	33k	2,200	51
Other setup	0.67	1.3%	100	.95	.97	45k	1,400	60
			100	.99	1.00	18k	3,400	66

5.74k–31.7k for HRNN. The other displayed baselines require 161k–107M evaluations and often miss the target. ANQI still verifies every candidate; its advantage comes from obtaining a high-recall candidate pool with less work. GIST1M has the largest gap: ANQI reaches recall/precision 0.96/0.997 at 4,400 QPS and can raise recall to 0.996 without reducing precision.

Inner product. At recall 0.99, ANQI reaches recall 0.99–1.0 and precision 1.0 at 510–23k QPS. The specialized baselines produce isolated points at 0.50–50 QPS; some reach exact recall but admit many false positives, with precision from 0.56 to 1.0. ANQI is 650–2,900 $\times$ faster than the fastest qualifying baseline on Music100. On MSMarcoWebSearch1M, the speedup grows from 10 $\times$ at $k = 10$ to 160 $\times$ at $k = 100$, while precision remains 1.0. ANQI fits the compact representation directly to the exact signed threshold matrix $T[o, k] = -ip_k(o)$; rank-specific residuals preserve its boundary detail.

Across both metrics, ANQI reduces candidate work before verification while maintaining precision of at least 0.997 at every selected point.

6.3 Cost and breakdown

Query throughput must be considered together with construction and storage. Against HRNN, ANQI builds 1.5–1.7 $\times$ faster on the three L_2 datasets and uses 2.4–9.0 $\times$ less index space. The smaller plain HNSW shared by SFT/RDT builds faster but shifts much more work to queries. ANQI’s lifted graph and verifier instead form one bundle for all three target ranks. On IP, ANQI makes the opposite trade-off: it builds in 550–880 seconds rather than 1.5–130 seconds, then provides 10–2,900 $\times$ higher selected-point throughput. The index is therefore intended for collections where a larger one-time build is amortized over sustained high-recall query traffic.

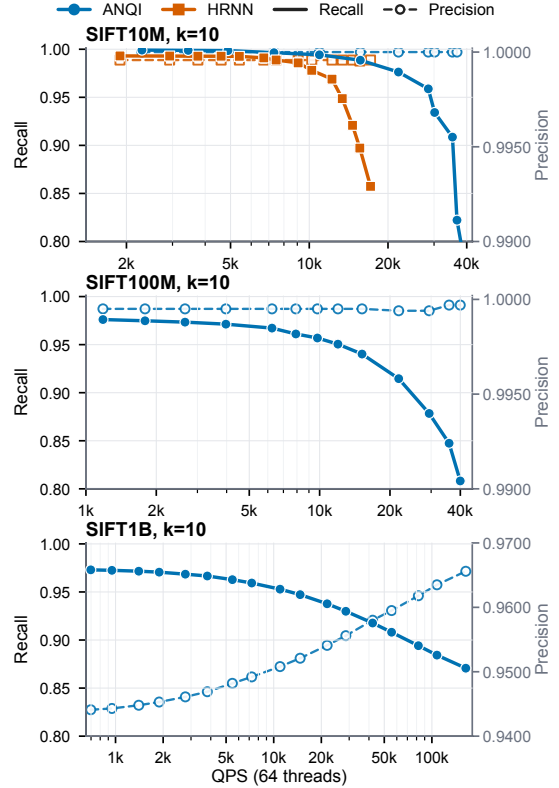


Figure 5: SIFT reverse-10NN scalability from 10M to 1B vectors. Filled and hollow traces show recall and precision; panels use independent QPS ranges. HRNN appears only at 10M. The 1B panel uses the earlier M6/Vamana scale prototype and an audited forward-derived proxy table.

Table 6 locates the remaining costs. The base-space AKNN warm start and graph build account for 74% and 23% of post-training construction; lift preparation, encoding, and setup total 3.1%. The separate 36-second model fit is 41% of the 89-second total in table 5. Model fitting and neighbor construction are therefore the main build-time targets.

At query time, graph traversal grows from 790 to 3,400 μ s as k or recall increases and accounts for 96–98% of CPU time. Verification remains at 30–66 μ s. The compact verifier is therefore neither the build nor query bottleneck after model fitting.

6.4 Scalability from 10M to 1B vectors

We extend the main benchmark by three orders of magnitude. Near recall 0.95, ANQI processes 29k QPS at 10M vectors, 12k at 100M, and 11k at 1B.

At 10M, ANQI achieves recall/precision 0.96/1.00 at 29k QPS, a 2.1 $\times$ gain over HRNN, while using fewer distance evaluations, building 16 $\times$ faster, and occupying a 10 $\times$ smaller index. At 100M it sustains 0.95/1.00 at 12k QPS with an 84 GB index. HRNN is omitted because its measured 10M artifact projects to an 810 GiB index and about 1.0 TiB of memory at 100M, beyond the available 88 GiB disk

Table 7: Representative SIFT points near recall 0.95. Q/D is QPS/mean complete online distance evaluations per query; B/I is query-index build time/index GB (10^9 bytes). The 1B R/P row uses the audited proxy protocol in fig. 5.

Scale	Method	R/P	Q/D	B/I
10M	ANQI	.96/1.00	29k/4.0k	160s/8.4
	HRNN	.95/1.00	13k/9.0k	2,600s/87
100M	ANQI	.95/1.00	12k/10k	2,900s/84
1B	ANQI	.95/0.95	11k/1.0k	1.3h/110

and 252 GiB host memory. Since it already fails this resource gate at 100M, we do not attempt it at 1B.

The 1B prototype reaches recall/precision 0.95/0.95 at 11k QPS with a 110 GB index built in 1.3 hours. Its forward top-10 proxy attains both ID and tie-aware recall@10 of 0.99 on 6,000 exact sampled queries. Across the displayed frontier, recall rises from 0.87 to 0.97 while precision remains between 0.94 and 0.97. Thus the audited scale result retains high recall and precision while keeping throughput above 10k QPS. The proxy audit and the separate 1B configuration are stated because the exact-ground-truth million-scale comparison and the billion-scale feasibility study answer different questions.

6.5 Local Radius Law in Euclidean Space

This experiment isolates the Euclidean low-rank term by setting $e_{o,k} = 0$ in eq. (19). A rank-8 PCA model is fitted to 120,000 exact DEEP1M threshold rows over ranks 1–500; all one million objects store eight coefficients computed from their complete 500-rank rows. Every evaluated rank $k = 10, \dots, 300$ participates in the representation, so the experiment measures in-range compression rather than unseen-rank extrapolation. Panel A evaluates threshold error over all base objects. Panel B applies the same reconstructed cutoffs in a full scan, removing graph-search misses from the diagnosis.

Without residual correction, mean relative error falls from 1.7% at $k = 10$ to 0.10% at $k = 300$, and the corresponding 99th percentiles fall from 6.1% to 0.48%. Underestimates account for 49–51% of the reconstructions, so the low-rank term has nearly balanced two-sided error. Full-scan recall/precision is 0.95/0.95 at $k = 10$, exceeds 0.98 at every all-query anchor with $k \geq 50$, and reaches 1.00/1.00 at $k = 300$. Eight coefficients therefore capture most cross-rank structure in 32 bytes per vector, while the remaining small- k boundary error motivates LRQ.

6.6 Why ANQI works: lifting and verifier ablations

Three ablations isolate graph geometry, verifier capacity, and the construction pivot while holding the remaining components fixed.

Graph geometry. Figure 8 changes only graph geometry. At the first measured point reaching recall 0.95, Lift reduces graph-distance work by factors of $1.4\times$ and $2.5\times$ for $k = 10$ and 100. At recall 0.99, the reductions grow to $2.8\times$ and $3.5\times$. The leftward separation is strongest in the high-recall tail. Both arms use M8+u8 with precision 1.0; an EXACT-F32 control has identical candidates and graph work and changes displayed recall by at most 0.002. The matched controls therefore locate the gain in navigation rather

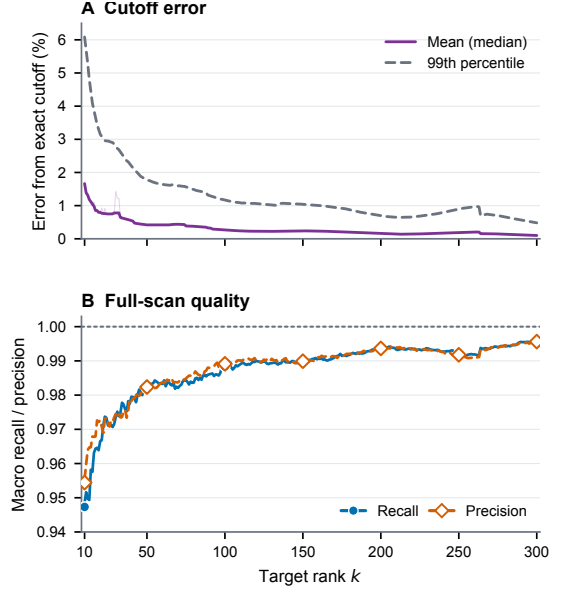


Figure 6: Standalone M8 Euclidean threshold modeling on DEEP1M for every integer $k = 10, \dots, 300$. Panel A reports mean and 99th-percentile cutoff error over all one million objects; the bold mean curve is a centered 7-rank visual guide over the faint raw trace. Panel B applies the same cutoffs in a full-base scan with no graph or search budget. Lines use 1,000 fixed queries; large symbols use all 10,000 queries at seven ranks.

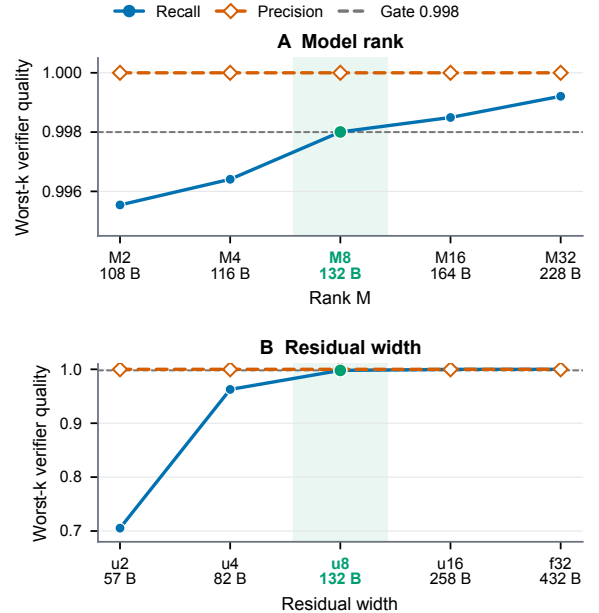


Figure 7: Compact-verifier ablation on DEEP1M. Panel A fixes u8 and varies model rank; Panel B fixes rank 8 and varies residual width. Tick labels include total verifier storage. M8+u8 is the lowest-storage setting meeting the selection criterion, at 132 B/vector.

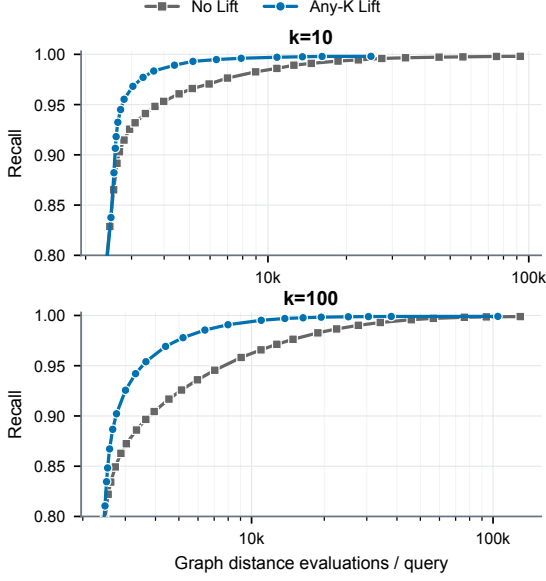


Figure 8: Controlled DEEP1M graph-geometry ablation for $k = 10$ and 100. Panels plot recall against graph-distance evaluations per query. No Lift uses the original graph; Lift uses the construction-rank reverse-aware graph. All other components are matched, and precision is 1.0 throughout.

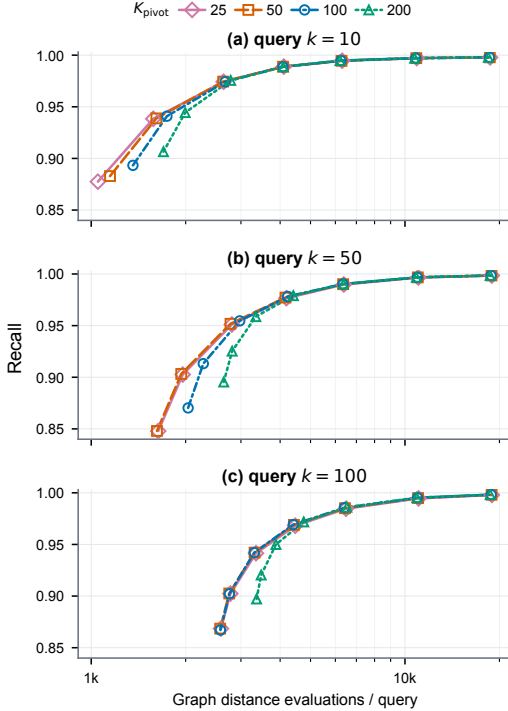


Figure 9: Construction-pivot robustness on DEEP1M. Each graph serves target ranks $k = 10, 50, 100$ with the target- k verifier envelope. Measured candidate counts follow the target rank, and precision is 1.0 throughout.

than verification. Lift adds only 1.2 seconds of construction time and about 4 MB to the index.

Construction pivot. We vary the construction pivot while retaining target- k candidate collection. Figure 9 shows that every tested pivot reaches recall 0.99 for all three query ranks, including $K_{\text{pivot}} = 25$ at $k = 100$. Within this evaluated range, the target- k envelope remains effective above the construction pivot. The curves nearly overlap at $k = 10$. At $k = 50$ and $L = 240$, pivot 25 falls just below the target at 0.9898 while the other pivots reach at least 0.9901, so its first sampled crossing moves to $L = 480$. By $k = 100$, all pivots cross recall 0.99 at the same sampled budget with about 11k graph-distance evaluations. Candidate counts remain near 11, 50, and 100 for the three target ranks, rather than growing to the construction pivot. Construction also varies little, at 66–69 seconds and about 0.60 GB. We use $K_{\text{pivot}} = 100$ because it matches the largest materialized target rank, not because the smaller pivots fail within the tested range.

Verifier capacity and residual width. Figure 7 reports controlled slices through the complete $M \in \{2, 4, 8, 16, 32\}$ by residual-width grid. With u8 fixed, worst- k recall rises from 0.996 at M2 to 0.998 at M8; M16 and M32 add less than 0.002 while increasing storage. With $M = 8$, u2/u4 residuals are too coarse, whereas u8 reaches 0.998/1.0 at 132 B/vector. More specifically, u2 and u4 retain precision 1.0 but reach only 0.71 and 0.96 worst-rank recall. U16 and float32 approach exact recall but require 258 and 432 B/vector. M8+u8 therefore preserves high-fidelity verification at about one third of the 400 B/vector dense threshold table. Candidate generation and distance counts are identical across these arms, so this is a storage-quality trade-off rather than a query-speed effect.

Together, the ablations attribute navigation savings to lifting and predicate fidelity to the compact verifier. They also show that one construction remains robust across the tested pivots and target ranks.

7 CONCLUSION

ANQI compiles object-specific reverse boundaries into a lifted graph for candidate routing and a compact, rank-aware verifier for final membership. This preserves the vector- and rank-specific semantics of RkNN while reusing a forward ANN graph. The Local Radius Law explains shared Euclidean cutoffs; signed projection quantiles support IP.

Ablations clarify their roles: lifting produces navigation savings, while LRQ retains decision fidelity at about one third of dense-table storage. Robustness across the tested L_2 pivots shows that the construction rank need not bound the query rank in this range. ANQI thus trades offline modeling for reusable threshold state and less online work.

Across five million-scale datasets, ANQI achieves multi-fold throughput gains at high recall with near-unit precision; it also constructs and queries a billion-vector SIFT index. Together, these results establish reusable reverse-threshold state as a scalable basis for approximate RkNN search over standard ANN graphs.

REFERENCES

- [1] Elke Achtert, Christian Böhm, Peer Kröger, Peter Kunath, Alexey Pryakhin, and Matthias Renz. 2006. Approximate reverse k-nearest neighbor queries in general metric spaces. In *Proceedings of the 15th ACM international conference on Information and knowledge management*. 788–789.
- [2] Elke Achtert, Christian Böhm, Peer Kröger, Peter Kunath, Alexey Pryakhin, and Matthias Renz. 2006. Efficient reverse k-nearest neighbor search in arbitrary metric spaces. In *Proceedings of the 2006 ACM SIGMOD international conference on Management of data*. 515–526.
- [3] Daichi Amagata and Takahiro Hara. 2021. Reverse maximum inner product search: How to efficiently find users who would like to buy my item?. In *Proceedings of the 15th ACM Conference on Recommender Systems*. 273–281.
- [4] Daichi Amagata and Takahiro Hara. 2023. Reverse maximum inner product search: Formulation, algorithms, and analysis. *ACM Transactions on the Web* 17, 4 (2023), 1–23.
- [5] Alexandr Andoni and Piotr Indyk. 2008. Near-optimal hashing algorithms for approximate nearest neighbor in high dimensions. *Commun. ACM* 51, 1 (2008), 117–122.
- [6] David Arthur and Steve Y Oudot. 2010. Reverse nearest neighbors search in high dimensions using locality-sensitive hashing. *arXiv preprint arXiv:1011.4955* (2010).
- [7] Franz Aurenhammer. 1987. Power diagrams: properties, algorithms and applications. *SIAM journal on computing* 16, 1 (1987), 78–96.
- [8] Artem Babenko and Victor Lempitsky. 2014. Additive quantization for extreme vector compression. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 931–938.
- [9] Yoram Bachrach, Yehuda Finkelstein, Ran Gilad-Bachrach, Liran Katzir, Noam Koenigstein, Nir Nave, and Ulrich Paquet. 2014. Speeding up the xbox recommender system using a euclidean transformation for inner-product spaces. In *Proceedings of the 8th ACM Conference on Recommender systems*. 257–264.
- [10] Zhengyang Bai, Peng Chen, and Mohamed Wahib. 2026. RT-RkNN: Reverse k Nearest Neighbor Queries as a Graphics Ray Casting Problem. *arXiv preprint arXiv:2605.26671* (2026).
- [11] Max Berrendorf, Felix Borutta, and Peer Kröger. 2019. k-distance approximation for memory-efficient RkNN retrieval. In *International Conference on Similarity Search and Applications*. Springer, 57–71.
- [12] Peter J Bickel and Leo Breiman. 1983. Sums of functions of nearest neighbor distances, moment bounds, limit theorems and a goodness of fit test. *The Annals of Probability* (1983), 185–214.
- [13] Guillaume Casanova, Elias Englmeier, Michael E Houle, Peer Kröger, Michael Nett, Erich Schubert, and Arthur Zimek. 2017. Dimensional testing for reverse k-nearest neighbor search. *Proceedings of the VLDB Endowment* 10, 7 (2017), 769–780.
- [14] Tingyang Chen, Cong Fu, Xiangyu Ke, Yunjun Gao, Yabo Ni, and Anxiang Zeng. 2025. Stitching inner product and euclidean metrics for topology-aware maximum inner product search. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2341–2350.
- [15] Tingyang Chen, Cong Fu, Kun Wang, Xiangyu Ke, Yunjun Gao, Wenchao Zhou, Yabo Ni, and Anxiang Zeng. 2025. Maximum inner product is query-scaled nearest neighbor. *arXiv preprint arXiv:2503.06882* (2025).
- [16] Yongjian Chen, Tao Guan, and Cheng Wang. 2010. Approximate nearest neighbor search by residual vector quantization. *Sensors* 10, 12 (2010), 11259–11273.
- [17] Otfried Cheong, Antoine Vigneron, and Juyoung Yon. 2011. Reverse nearest neighbor queries in fixed dimension. *International Journal of Computational Geometry & Applications* 21, 02 (2011), 179–188.
- [18] Mayur Datar, Nicole Immorlica, Piotr Indyk, and Vahab S Mirrokni. 2004. Locality-sensitive hashing scheme based on p-stable distributions. In *Proceedings of the twentieth annual symposium on Computational geometry*. 253–262.
- [19] Wei Dong, Charikar Moses, and Kai Li. 2011. Efficient k-nearest neighbor graph construction for generic similarity measures. In *Proceedings of the 20th international conference on World wide web*. 577–586.
- [20] Carl Eckart and Gale Young. 1936. The approximation of one matrix by another of lower rank. *Psychometrika* 1, 3 (1936), 211–218.
- [21] Elena Facco, Maria d’Errico, Alex Rodriguez, and Alessandro Laio. 2017. Estimating the intrinsic dimension of datasets by a minimal neighborhood information. *Scientific reports* 7, 1 (2017), 12140.
- [22] Cong Fu and Deng Cai. 2016. Efanna: An extremely fast approximate nearest neighbor search algorithm based on knn graph. *arXiv preprint arXiv:1609.07228* (2016).
- [23] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2017. Fast approximate nearest neighbor search with the navigating spreading-out graph. *arXiv preprint arXiv:1707.00143* (2017).
- [24] Yujian Fu, Cheng Chen, Yao Chen, Weng-Fai Wong, and Bingsheng He. 2025. Vista: Vector indexing and search for large-scale imbalanced datasets. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, 543–556.
- [25] Keinosuke Fukunaga and L Hostetler. 1973. Optimization of k nearest neighbor density estimates. *IEEE Transactions on information theory* 19, 3 (1973), 320–326.
- [26] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2013. Optimized product quantization for approximate nearest neighbor search. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2946–2953.
- [27] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating large-scale inference with anisotropic vector quantization. In *International Conference on Machine Learning*. PMLR, 3887–3896.
- [28] Kiana Hajebi, Yasin Abbasi-Yadkori, Hossein Shahbazi, and Hong Zhang. 2011. Fast approximate nearest-neighbor search with k-nearest neighbor graph. In *IJCAI Proceedings-International Joint Conference on Artificial Intelligence*, Vol. 22. 1312.
- [29] Ben Harwood and Tom Drummond. 2016. Fanng: Fast approximate nearest neighbour graphs. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 5713–5722.
- [30] Qiang Huang, Yanhao Wang, and Anthony KH Tung. 2023. Sah: Shifting-aware asymmetric hashing for reverse k maximum inner product search. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 4312–4321.
- [31] Piotr Indyk and Rajeev Motwani. 1998. Approximate nearest neighbors: towards removing the curse of dimensionality. In *Proceedings of the thirtieth annual ACM symposium on Theory of computing*. 604–613.
- [32] Masajiro Iwasaki. 2016. Pruned bi-directed k-nearest neighbor graph for proximity search. In *International Conference on Similarity Search and Applications*. Springer, 20–33.
- [33] Masajiro Iwasaki and Daisuke Miyazaki. 2018. Optimization of indexing based on k-nearest neighbor graph for proximity search in high-dimensional data. *arXiv preprint arXiv:1810.07355* (2018).
- [34] Subhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. Diskann: Fast accurate billion-point nearest neighbor search on a single node. *Advances in neural information Systems* 32 (2019).
- [35] Herve Jegou, Matthijs Douze, and Cordelia Schmid. 2010. Product quantization for nearest neighbor search. *IEEE transactions on pattern analysis and machine intelligence* 33, 1 (2010), 117–128.
- [36] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE transactions on big data* 7, 3 (2019), 535–547.
- [37] Yannis Kalantidis and Yannis Avrithis. 2014. Locally optimized product quantization for approximate nearest neighbor search. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2321–2328.
- [38] Flip Korn and Suresh Muthukrishnan. 2000. Influence sets based on reverse nearest neighbor queries. *ACM Sigmod Record* 29, 2 (2000), 201–212.
- [39] Elizaveta Levina and Peter Bickel. 2004. Maximum likelihood estimation of intrinsic dimension. *Advances in neural information processing systems* 17 (2004).
- [40] Jie Liu, Xiao Yan, Xinyan Dai, Zhirong Li, James Cheng, and Ming-Chang Yang. 2020. Understanding and improving proximity graph based maximum inner product search. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 34. 139–146.
- [41] Don O Loftsgaarden and Charles P Quesenberry. 1965. A nonparametric estimate of a multivariate density function. *The Annals of Mathematical Statistics* 36, 3 (1965), 1049–1051.
- [42] Yu A Malkov and Dmitry A Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence* 42, 4 (2018), 824–836.
- [43] Magdalen Dobson Manohar, Taekseung Kim, and Guy E Blelloch. 2025. Range retrieval with graph-based indices. *arXiv preprint arXiv:2502.13245* (2025).
- [44] Magdalen Dobson Manohar, Zheqi Shen, Guy Blelloch, Laxman Dhulipala, Yan Gu, Harsha Vardhan Simhadri, and Yihan Sun. 2024. Parlayann: Scalable and deterministic parallel graph-based approximate nearest neighbor search algorithms. In *Proceedings of the 29th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*. 270–285.
- [45] Stanislav Morozov and Artem Babenko. 2018. Non-metric similarity graphs for maximum inner product search. *Advances in Neural Information Processing Systems* 31 (2018).
- [46] Marius Muja and David G Lowe. 2009. Fast approximate nearest neighbors with automatic algorithm configuration. In *International conference on computer vision theory and applications*, Vol. 1. SciTePress, 331–340.
- [47] Behnam Neyshabur and Nathan Srebro. 2015. On symmetric and asymmetric lshs for inner product search. In *International Conference on Machine Learning*. PMLR, 1926–1934.
- [48] Sandra Obermeier, Max Berrendorf, and Peer Kröger. 2020. Memory-Efficient RkNN Retrieval by Nonlinear k-Distance Approximation. In *2020 IEEE/WIC/ACM International Joint Conference on Web Intelligence and Intelligent Agent Technology (WI-IAT)*. IEEE, 387–390.
- [49] Anshumali Shrivastava and Ping Li. 2014. Asymmetric LSH (ALSH) for sublinear time maximum inner product search (MIPS). *Advances in neural information processing systems* 27 (2014).

- [50] Anshumali Shrivastava and Ping Li. 2014. Improved asymmetric locality sensitive hashing (ALSH) for maximum inner product search (MIPS). *arXiv preprint arXiv:1410.5410* (2014).
- [51] Amit Singh, Hakan Ferhatoşmanoglu, and Ali Şaman Tosun. 2003. High dimensional reverse nearest neighbor queries. In *Proceedings of the twelfth international conference on Information and knowledge management*. 91–98.
- [52] Yitong Song, Kai Wang, Bin Yao, Zhida Chen, Jiong Xie, and Feifei Li. 2024. Efficient Reverse k Approximate Nearest Neighbor Search Over High-Dimensional Vectors. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 4262–4274.
- [53] Shulong Tan, Zhaozhuo Xu, Weijie Zhao, Hongliang Fei, Zhixin Zhou, and Ping Li. 2021. Norm adjusted proximity graph for fast inner product retrieval. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 1552–1560.
- [54] Yufei Tao, Dimitris Papadias, Xiang Lian, and Xiaokui Xiao. 2007. Multidimensional reverse k NN search. *The VLDB Journal* 16, 3 (2007), 293–316.
- [55] Wenxuan Xia, Mingyu Yang, Wentao Li, and Wei Wang. 2026. HRNN: A Hybrid Graph Index for Approximate Reverse k -Nearest Neighbor Search on High-Dimensional Vectors. *arXiv preprint arXiv:2606.03225* (2026).
- [56] Xiao Yan, Jinfeng Li, Xinyan Dai, Hongzhi Chen, and James Cheng. 2018. Norm-ranging LSH for maximum inner product search. *Advances in Neural Information Processing Systems* 31 (2018).
- [57] Ting Zhang, Chao Du, and Jingdong Wang. 2014. Composite quantization for approximate nearest neighbor search. In *International Conference on Machine Learning*. PMLR, 838–846.
- [58] Xi Zhao, Bolong Zheng, Xiaomeng Yi, Xiaofan Luan, Charles Xie, Xiaofang Zhou, and Christian S Jensen. 2023. FARGO: Fast maximum inner product search via global multi-probing. *Proceedings of the VLDB Endowment* 16, 5 (2023), 1100–1112.
- [59] Zhixin Zhou, Shulong Tan, Zhaozhuo Xu, and Ping Li. 2019. Möbius transformation for fast inner product search on graph. *Advances in Neural Information Systems* 32 (2019).

A PROOFS AND SUPPLEMENTARY ANALYSIS

This appendix gives the complete algebra behind the statements in Sections 4 and 5. We use exact thresholds and real arithmetic unless stated otherwise. Thus the proofs separate identities that hold for the represented predicate from the graph-coverage property, which is evaluated empirically by the finite ANN traversal.

A.1 Exact lifting and rank sharing

The first result establishes the role of lifting in the system. It does not change which objects satisfy the reverse predicate. It changes only the representation so that a forward Euclidean search order can find those objects, after which the same predicate can be applied to the retrieved candidates.

PROOF OF THEOREM 1. We first verify the predicate in the original score space. For L_2 , (9) gives

$$\begin{aligned} \|q - o\|^2 \leq r_k(o)^2 &\iff \|q\|^2 + \|o\|^2 - 2 \cdot \langle q, o \rangle \leq r_k(o)^2 \\ &\iff \langle q, o \rangle - \frac{1}{2} \cdot (\|o\|^2 - r_k(o)^2) \geq \frac{1}{2} \cdot \|q\|^2. \end{aligned}$$

The left-hand side is exactly $s_k(q, o) = \langle q^\ell, o_k^\ell \rangle$ for the L_2 row in (11), and the right-hand side is $\tau(q)$. For inner product, the same lifted inner product is $s_k(q, o) = \langle q, o \rangle - \text{ip}_k(o)$, so

$$o \in A_k(q) \iff s_k(q, o) \geq 0 = \tau(q).$$

Thus, for either metric,

$$o \in A_k(q) \iff s_k(q, o) \geq \tau(q).$$

Now consider the norm-equalized vectors in (13). The last coordinate is real because $M_\star$ is at least the largest norm of the lifted indexed vectors. Hence $\|\bar{o}_k\|^2 = \|o_k^\ell\|^2 + M_\star^2 - \|o_k^\ell\|^2 = M_\star^2$. Using $\bar{q} = [q^\ell, 0]$ and expanding the squared distance gives

$$\begin{aligned} \|\bar{q} - \bar{o}_k\|^2 &= \|\bar{q}\|^2 + \|\bar{o}_k\|^2 - 2 \cdot \langle \bar{q}, \bar{o}_k \rangle \\ &= \|q^\ell\|^2 + M_\star^2 - 2 \cdot \langle q^\ell, o_k^\ell \rangle \\ &= \|q^\ell\|^2 + M_\star^2 - 2 \cdot s_k(q, o). \end{aligned}$$

Since the first two terms are independent of o , a larger score is equivalent to a smaller Euclidean distance. Substituting $s_k(q, o) \geq \tau(q)$ into this identity yields

$$\begin{aligned} s_k(q, o) \geq \tau(q) &\iff \|\bar{q} - \bar{o}_k\|^2 \leq \|q^\ell\|^2 + M_\star^2 - 2 \cdot \tau(q) \\ &\iff \|\bar{q} - \bar{o}_k\|^2 \leq \rho_k(q)^2, \end{aligned}$$

which proves the lifted-ball equivalence and the score-ordering claim. Finally, for any candidate set $C(q)$, applying this pointwise equivalence only to vectors in $C(q)$ gives

$$\{o \in C(q) : \|\bar{q} - \bar{o}_k\|^2 \leq \rho_k(q)^2\} = C(q) \cap A_k(q).$$

This is a statement about filtering the supplied candidates; it does not claim that a finite graph traversal necessarily supplies every member of $A_k(q)$. $\square$

PROOF OF PROPOSITION 4. For the L_2 case, define $N_o(r) = |\{x \in \mathcal{X} \setminus \{o\} : \|x - o\| \leq r\}|$. By definition, $r_k(o)$ is a k th-neighbor radius, so $N_o(r_k(o)) \geq k$. If $K \geq k$, the K th order statistic cannot be smaller than the k th one:

$$r_k(o) \leq r_K(o).$$

Therefore, whenever $\|q - o\| \leq r_k(o)$, we also have $\|q - o\| \leq r_K(o)$, proving $A_k(q) \subseteq A_K(q)$.

For inner product, let the scores $\{\langle x, o \rangle : x \neq o\}$ be sorted in nonincreasing order. The k th order statistic satisfies $\text{ip}_k(o) \geq \text{ip}_K(o)$ for $K \geq k$. Hence

$$\langle q, o \rangle \geq \text{ip}_k(o) \implies \langle q, o \rangle \geq \text{ip}_K(o),$$

which proves the same set containment. Theorem 1 maps each of these pointwise predicates to its lifted-ball predicate, so the construction-rank acceptance region at K is a valid exact candidate envelope for every represented target rank $k \leq K$. The proposition does not by itself provide a finite-beam graph-coverage guarantee. $\square$

The shared-rank result is therefore a candidate-generation statement: a graph built for a larger construction rank contains the exact acceptance region of every smaller represented rank. It says nothing about how well a finite-beam traversal covers that region; that is the quantity measured in the cross-pivot experiments.

PROOF OF LEMMA 1. By definition of the construction-rank score,

$$s_{K_{\text{pivot}}}(q, v) = \langle q, v \rangle - \frac{1}{2} \cdot (\|v\|^2 - t_{K_{\text{pivot}}}(v)).$$

Adding the correction and grouping the two terms that contain $t_{K_{\text{pivot}}}(v)$ gives

$$\begin{aligned} s_{K_{\text{pivot}}}(q, v) &+ \frac{1}{2} \cdot (\tilde{t}_k(v) - t_{K_{\text{pivot}}}(v)) \\ &= \langle q, v \rangle - \frac{1}{2} \cdot \|v\|^2 + \frac{1}{2} \cdot t_{K_{\text{pivot}}}(v) + \frac{1}{2} \cdot \tilde{t}_k(v) - \frac{1}{2} \cdot t_{K_{\text{pivot}}}(v) \\ &= \langle q, v \rangle - \frac{1}{2} \cdot (\|v\|^2 - \tilde{t}_k(v)) \\ &= s_{\tilde{t}_k}(q, v). \end{aligned}$$

The identity is algebraic and holds for any represented rank, including a rank larger than K_{pivot} . If the reconstruction is exact, $\tilde{t}_k(v) = t_k(v)$, then the final expression is the exact target- k reverse-aware score by (9). What remains empirical beyond the construction pivot is whether the graph traversal reaches all objects in the new target-rank envelope, not whether this score correction is algebraically valid. $\square$

A.2 Euclidean local-radius analysis

The next derivation explains why neighboring Euclidean ranks can share a compact model. Under a locally stable density and intrinsic dimension, the squared radius is approximately a power of the rank. Expanding the small deviation from a reference exponent produces shared rank functions and object-specific coefficients, which is the structural motivation for the low-rank verifier.

PROOF OF PROPOSITION 1. Under the local ball-mass model,

$$N_o(r) = \rho(o) \cdot V_{d(o)} \cdot r^{d(o)}.$$

At the k th neighbor radius, $N_o(r_k(o)) = k$, so

$$r_k(o)^{d(o)} = \frac{k}{\rho(o) \cdot V_{d(o)}}.$$

Raising both sides to the power $2/d(o)$ gives

$$\begin{aligned} r_k(o)^2 &= \left(\frac{k}{\rho(o) \cdot V_{d(o)}} \right)^{2/d(o)} \\ &= [\rho(o) \cdot V_{d(o)}]^{-2/d(o)} \cdot k^{2/d(o)} \\ &= \gamma(o) \cdot k^{\nu(o)}. \end{aligned}$$

with $\nu(o) = 2/d(o)$ and $\gamma(o) = [\rho(o) \cdot V_{d(o)}]^{-\nu(o)}$.

Write $\nu(o) = \nu_0 + \beta(o)$. Then

$$k^{\nu(o)} = k^{\nu_0 + \beta(o)} = k^{\nu_0} \cdot \exp(\beta(o) \cdot \ln k).$$

For $x = \beta(o) \cdot \ln k$, Taylor's theorem gives

$$e^x = \sum_{m=0}^p \frac{x^m}{m!} + R_p(x), \quad |R_p(x)| \leq e^{|x|} \frac{|x|^{p+1}}{(p+1)!}.$$

Multiplying by $\gamma(o) \cdot k^{\nu_0}$ and inserting $x = \beta(o) \cdot \ln k$ produces

$$r_k(o)^2 = \sum_{m=0}^{\infty} \underbrace{\frac{\gamma(o) \cdot \beta(o)^m}{m!}}_{f_m(o)} \underbrace{k^{\nu_0} \cdot (\ln k)^m}_{\phi_m(k)}.$$

The same multiplication applied to the Taylor remainder gives

$$|R_p(o, k)| \leq \gamma(o) \cdot k^{\nu_0} \cdot e^{|\beta(o) \cdot \ln k|} \cdot \frac{|\beta(o) \cdot \ln k|^{p+1}}{(p+1)!},$$

The factorization shows why a small shared rank basis can be useful when $|\beta(o) \cdot \ln k|$ is moderate. It is an object-local approximation: changes in density, curvature, or intrinsic dimension can invalidate one exponent over a wider rank interval, and no claim about the ambient dimension D is needed for the derivation. $\square$

PROOF OF COROLLARY 1. Applying Proposition 1 at k and K gives

$$r_k(o)^2 = \gamma(o) \cdot k^{\nu(o)}, \quad r_K(o)^2 = \gamma(o) \cdot K^{\nu(o)}.$$

Because $\gamma(o) > 0$, division cancels the object-specific intensity and volume factor:

$$\frac{r_k(o)^2}{r_K(o)^2} = \left(\frac{k}{K} \right)^{\nu(o)}.$$

The local mass equation also gives $N_o(r_k(o)) = k$ and $N_o(r_K(o)) = K$. The shell between the two radii therefore has mass

$$N_o(r_k(o)) - N_o(r_K(o)) = k - K.$$

The result describes the local model's expected shell mass; it does not assert that a finite sampled dataset has exactly uniform shell placement at every rank. $\square$

A.3 IP quantiles and one-sided verification

The IP result has a different interpretation. Inner-product thresholds are directional projection quantiles, not Euclidean radii. The identity below explains the exact threshold being modeled, while the low-rank-plus-LRQ representation remains an empirical compression of that signed threshold matrix.

PROOF OF PROPOSITION 2. Suppose first that $o \neq 0$ and define $u_o = o/\|o\|$. For every base vector x , bilinearity of the inner product gives

$$\langle x, o \rangle = \langle x, \|o\|u_o \rangle = \|o\| \cdot \langle x, u_o \rangle.$$

The multiplier $\|o\|$ is positive and is the same for every x , so it preserves the ordering of all projection scores. Consequently, the k th largest inner product equals the same positive multiplier times the k th largest projection:

$$\text{ip}_k(o) = \|o\| \cdot \pi_k(o).$$

The reverse predicate becomes

$$\begin{aligned} \langle q, o \rangle \geq \text{ip}_k(o) &\iff \|o\| \cdot \langle q, u_o \rangle \geq \|o\| \cdot \pi_k(o) \\ &\iff \langle q, u_o \rangle \geq \pi_k(o), \end{aligned}$$

where the second equivalence divides by the positive number $\|o\|$. Since $T[o, k] = -\text{ip}_k(o)$ by definition, the signed threshold row is $T[o, k] = -\|o\| \cdot \pi_k(o)$. If $o = 0$, every inner product with o is zero; all order statistics and the signed threshold row are therefore zero, which is the stated special case. $\square$

PROOF OF PROPOSITION 3. Assume first that the reconstruction is restrictive, so $\hat{t}_k(o) \leq t_k(o)$ for every o . If an object is accepted by the represented verifier, then

$$\kappa(q, o) \leq \hat{t}_k(o) \leq t_k(o),$$

and it is therefore also in the exact answer. This proves $\hat{A}_k(q) \subseteq A_k(q)$ and rules out false positives. If an exact member is rejected, it satisfies the exact acceptance inequality but violates the represented one, namely

$$\hat{t}_k(o) < \kappa(q, o) \leq t_k(o).$$

Thus every false negative lies in the displayed boundary band, and objects outside that band have the same decision under both thresholds.

For the expansive case, $\hat{t}_k(o) \geq t_k(o)$. Every exact member then obeys

$$\kappa(q, o) \leq t_k(o) \leq \hat{t}_k(o),$$

so $A_k(q) \subseteq \hat{A}_k(q)$ and verification cannot introduce false negatives. The only possible errors are objects in the opposite boundary band $\{o : t_k(o) < \kappa(q, o) \leq \hat{t}_k(o)\}$, which can be false positives. $\square$

A.4 Supplementary pooled-recall bound

The following calculation translates a relative error in squared Euclidean radius into a conservative recall statement under the same local model. It applies to the verifier component only; graph candidates that were never reached are outside this bound.

Assume a restrictive L_2 reconstruction and let

$$\epsilon_k = \max_o \frac{r_k(o)^2 - \hat{r}_k(o)^2}{r_k(o)^2} < 1.$$

Then, for every object,

$$\hat{r}_k(o)^2 \geq (1 - \epsilon_k) \cdot r_k(o)^2, \quad \hat{r}_k(o) \geq \sqrt{1 - \epsilon_k} r_k(o).$$

Under $N_o(r) = \rho(o) \cdot V_{d(o)} \cdot r^{d(o)}$, the fraction of the exact local ball retained by the represented ball is at least

$$\begin{aligned} \frac{N_o(\hat{r}_k(o))}{N_o(r_k(o))} &= \left(\frac{\hat{r}_k(o)}{r_k(o)} \right)^{d(o)} \\ &\geq (1 - \epsilon_k)^{d(o)/2} \\ &\geq (1 - \epsilon_k)^{d_{\max}/2}, \end{aligned}$$

where $d_{\max} = \max_o d(o)$ and $0 < 1 - \epsilon_k < 1$. Averaging these per-object retention ratios with the exact answer-set sizes as weights gives the same lower bound for pooled recall. This is a verifier-side bound under the local model; it does not account for candidates that the finite graph traversal fails to visit. Thus

$$\text{Recall}_{\text{pool}} \geq (1 - \epsilon_k)^{d_{\max}/2}. \quad (31)$$

A.5 Quantization width and threshold error

This section connects the residual codebook to the previous recall bound. The codebook spacing is a direct knob: a smaller spacing narrows the threshold boundary where floor decoding can reject an exact member, and therefore gives a stronger model-based recall bound.

The one-sided property of floor decoding can be related directly to the spacing of the residual codebook. Let the ordered entries of $\mathcal{B}_k$ be $b_{0,k} < \dots < b_{j_k-1,k}$, and assume that the codebook covers every residual encoded at rank k . Define its maximum adjacent spacing by

$$\Delta_k = \max_{0 \leq j < j_k-1} (b_{j+1,k} - b_{j,k}).$$

For a residual in the interval $[b_{j,k}, b_{j+1,k})$, floor decoding selects $b_{j,k}$, so the following bound holds:

$$0 \leq e_{o,k} - \tilde{e}_{o,k} < \Delta_k, \quad 0 \leq t_k(o) - \hat{t}_k(o) < \Delta_k. \quad (32)$$

The second inequality follows by subtracting the shared mean and basis terms in (19) and (20). Residuals equal to the largest codebook endpoint have zero error; including that endpoint handles the boundary case omitted by the open interval above.

For positive Euclidean thresholds, define $r_{k,\min}^2 = \min_o r_k(o)^2$. If $\Delta_k < r_{k,\min}^2$, then the restrictive relative error in (31) obeys

$$\epsilon_k \leq \frac{\Delta_k}{r_{k,\min}^2},$$

and the local ball-mass model gives the conservative pooled bound

$$\text{Recall}_{\text{pool}} \geq \left(1 - \frac{\Delta_k}{r_{k,\min}^2}\right)^{d_{\max}/2}. \quad (33)$$

More locally, an object with radius $r_k(o)$ retains at least the fraction $(1 - \Delta_k/r_k(o)^2)^{d(o)/2}$ of its modeled ball. These bounds explain the role of residual width without asserting that a finite dataset exactly follows the local model; the measured verifier curves provide the empirical check.

A.6 Candidate coverage and verifier retention

The next identity states the two-part interpretation of end-to-end recall. A query can lose an exact answer because the graph never returns it, or because the verifier rejects a returned candidate. The factors below name those two events without assuming that they are statistically independent.

The two error sources in the indexed query admit an exact recall decomposition. For a query with nonempty exact answer $A = A_k(q)$, let $C = C(q)$ be the graph candidate set and let $\hat{R} = \hat{R}_k(q) \subseteq C$ be

the verified output. Then

$$\begin{aligned} \text{Recall}(q) &= \frac{|A \cap \hat{R}|}{|A|} \\ &= \underbrace{\frac{|A \cap C|}{|A|}}_{\text{candidate coverage}} \underbrace{\frac{|\hat{R}|}{|A \cap C|}}_{\text{verifier retention}}, \end{aligned} \quad (34)$$

where the second factor is defined as zero when $|A \cap C| = 0$. This identity is purely set-theoretic. The first factor is determined by the lifted graph traversal; the second is determined by the represented threshold on the candidates that were actually reached. Under exact thresholds, verifier retention is one, so recall is limited only by graph coverage. Under a restrictive floor-decoded threshold, candidate coverage is unchanged while verifier retention can drop only for the boundary band in Proposition 3.

This factorization also explains the experimental separation: the lift and graph ablations target the first factor, while the M-only and verifier-capacity controls target the second. It is not a claim that the two factors are statistically independent; it is an exact decomposition for each query.

A.7 Conditional completeness beyond the construction pivot

Finally, we make the scope of the cross-pivot argument explicit. The score correction is exact for a represented target rank, but exact end-to-end recovery additionally requires the graph traversal to cover the corresponding target envelope. The next proposition states that implication under that explicit condition.

The target-rank correction gives a useful conditional statement for cross-pivot queries. For the L_2 path, let $E_k(q)$ denote the target envelope evaluated by the corrected score in (27). In the following proposition, “exhaustive” refers to the graph traversal, not merely to the verifier scan.

PROPOSITION 5 (CONDITIONAL CROSS-PIVOT COMPLETENESS). *Assume exact thresholds and exact arithmetic. If the traversal for (q, k) is exhaustive over $E_k(q)$ and reaches every indexed object in that envelope, then the verifier returns the exact target- k answer, including when $k > K_{\text{pivot}}$.*

PROOF OF PROPOSITION 5. By Lemma 1, the corrected construction-rank score is exactly $s_{t_k}(q, o)$ when the target threshold is represented without error. Equation (9) therefore gives

$$E_k(q) = \{o : s_{t_k}(q, o) \geq \tfrac{1}{2} \cdot \|q\|^2\} = A_k(q).$$

The exhaustive-traversal assumption supplies every member of $A_k(q)$ to the verifier. Because the verifier uses the exact same threshold predicate, it accepts every such member and rejects every non-member in the visited envelope. Hence its output is exactly $A_k(q)$. The proof does not assert that a finite-beam ANN traversal is exhaustive; that coverage condition is the empirical quantity tested by the cross-pivot experiments. $\square$